

Code Your Own Game

JavaScript Curriculum for Kids and Teens

A five-week, step-by-step guide from your first line of code to a playable browser game built with HTML Canvas, CSS and JavaScript

Week 1 JavaScript Foundations & Setup

Week 2 Control Flow & Loops

Week 3 Functions, Data & the DOM

Week 4 Game Mechanics & Canvas

Week 5 Polish, Debug & Present

Contents

How To Use This Guide	4
The Big Goal	4
What You Will Need For The Whole Course	5
Week 1: JavaScript Foundations & Setup	6
Welcome to Programming!	6
What Is JavaScript, and Why JavaScript?	7
Setting Up Your Workshop	8
Your First Web Page and Your First JavaScript	10
Variables: Labeled Storage Boxes	12
Data Types: What Kind of Value Is It?	14
Comments: Notes to Humans	16
Doing Math With Operators	17
Getting Input and Converting Types	18
Simple Mini Projects	19
Debugging Basics	20
Week 1 Recap	21
Check Your Understanding	22
Week 2: Control Flow & Loops	23
Comparison Operators	23
Making Decisions With if, else if, and else	24
Boolean Logic: The Brains of the Game	25
Loops: Repeating Without Retyping	27
Randomness: Making Games Unpredictable	29
Putting It All Together: A Number Guessing Game	30
Another Example: Rock, Paper, Scissors	32
Week 2 Recap	33
Check Your Understanding	33
Week 3: Functions, Data & the DOM	34
Functions: Creating Your Own Commands	34
Arrays: Lists of Things	37
Objects: Describing One Thing in Detail	39
HTML and CSS Essentials for Game Builders	40

The DOM: JavaScript Meets the Page	42
Project: Catch the Bug (Your First Mini Game)	44
Week 3 Recap	49
Check Your Understanding	49
Week 4: Game Mechanics & Canvas	50
The Core Game Loop	50
Planning Our Game: Alien Dodge	51
Meet the Canvas	52
Game Objects: Building the Ship	55
Making It Move: The Animation Loop	55
Keyboard Controls	57
Falling Aliens	58
Collision Detection	60
Score, Lives, and Game States	61
The Complete Game: Alien Dodge	63
Tuning Your Game	68
Week 4 Recap	69
Check Your Understanding	69
Week 5: Polish, Debug & Present	70
From "It Works" to "It's Great"	70
Testing: Try to Break Your Own Game	70
User Experience: How the Game Feels	71
Polishing Alien Dodge	72
Debugging Like a Professional	77
Sharing Your Game	79
Presentation Day	80
Your Next Game	80
Week 5 Recap	83
Check Your Understanding	83
Quick Reference Cheat Sheet	84
Full Course Glossary	85

How To Use This Guide

Think of this guide like a recipe book, except instead of cooking food, we are cooking up a game! Each week is a new step in the recipe, and every step builds on the one before it. Don't skip ahead: Week 4's game mechanics only make sense once you know the ingredients from Weeks 1 through 3.

Here is what you will find inside every lesson:

- A short explanation of the new idea, using everyday comparisons.
- A piece of real JavaScript code that shows the idea in action.
- A box called "Here's What's Happening" that walks through the code one line at a time, in plain English.
- A "Mini Challenge" so you can practice the idea yourself before moving on.
- Occasional "Tip" boxes with shortcuts, warnings, or fun facts.

Here's What's Happening:

Whenever you see this box, it means we are about to slow down and look at code super closely. We will never just show you code and expect you to guess what it does. Every single line gets explained, the same way a patient friend would explain it to you while sitting next to you at the computer.

Tip:

Keep a notebook (real paper or digital) next to you as you go through this guide. Jot down new words like variable, function, or loop. You'll be surprised how quickly they become part of your everyday vocabulary!

The Big Goal

Every single lesson in this guide points toward one destination: a finished, playable game that runs in a web browser, built with your own hands and your own code. By the end of this guide, you will apply everything you have learned to create interactive browser-based games using JavaScript together with HTML Canvas (for drawing and animation) and CSS (for styling and layout). Here is the roadmap:

1. **Week 1:** Set up your tools and learn the basic building blocks of JavaScript: variables, data types, operators, and simple input and output.
2. **Week 2:** Teach your programs to make decisions and repeat actions, using conditionals, loops, and randomness.
3. **Week 3:** Organize your code into reusable functions, store game data in arrays and objects, and bring a web page to life with the DOM, CSS, and events. You will finish the week with your first mini game, built with only HTML, CSS, and JavaScript.
4. **Week 4:** Meet the HTML Canvas and combine everything into a real game loop with keyboard controls, collisions, scoring, and win and lose conditions. You will build a full game called Alien

Dodge.

5. **Week 5:** Polish your game, hunt down bugs like a professional, save a high score in the browser, and get ready to show it off to family and friends.

Ready? Grab your computer, take a deep breath, and let's start coding!

What You Will Need For The Whole Course

- A computer (Windows, macOS, or Linux) with a modern web browser such as Chrome, Edge, Firefox, or Safari.
- A free code editor. We will use Visual Studio Code in Week 1.
- About 45 to 60 minutes of quiet, focused time for each lesson.
- A notebook or blank document for new vocabulary.
- Optional: a parent, teacher, or friend who can play-test your games and give you feedback.

Tip:

You never need to buy anything or install anything complicated for this course. Every web browser already contains a JavaScript engine, which means the computer you are using right now can already run everything in this guide.

WEEK 1

JavaScript Foundations & Setup

Building your workshop and learning the first tools

Welcome to Programming!

Welcome! You are about to start one of the most exciting journeys you can take on a computer: learning to speak its language. Over the next five weeks, you will go from typing your very first line of code to building a complete, playable video game that anyone can open in a web browser.

You don't need any experience to start. You don't need to be a math genius, and you don't need a fancy computer. You just need curiosity, a little patience, and a willingness to try things, make mistakes, and try again.

What is Programming?

Programming is the process of giving a computer a clear set of instructions to perform specific tasks and solve problems. It allows us to create things that didn't exist before: games, mobile apps, websites, robots, and even artificial intelligence. Every app on your phone, every game you've played, and every website you've visited began as a collection of instructions written by a programmer.

Programmers are the creative minds behind almost everything you interact with on a screen. They carefully think through problems such as: How should the character move? What should happen when a player presses a button? How does the game know when the player has won? They then write step-by-step instructions that tell the computer exactly how to make those things happen.

Tip:

Mistakes are not failures in programming; they are opportunities to learn. Professional programmers make mistakes every day. The difference between a beginner and an expert isn't that experts never make mistakes. It's that they have more experience finding, understanding, and fixing them.

What Is Coding?

Coding is the process of writing instructions that tell a computer what to do using a programming language such as JavaScript. It is the act of turning ideas into code that a computer can read and execute.

However, coding is only one part of programming. Programming is the complete process of creating a computer program. It involves several important steps:

1. **Planning:** Thinking about the problem and deciding what the program should do.
2. **Designing:** Organizing how the program will work and how different parts will fit together.

3. **Coding:** Writing the instructions using a programming language.
4. **Testing:** Running the program to check whether it works correctly.
5. **Debugging:** Finding and fixing mistakes (called bugs).
6. **Improving:** Adding new features and making the program better over time.

Tip:

Programming is the complete journey of creating a computer program, while coding is the important step where we write the instructions that bring our ideas to life. Every programmer is a coder, but good programming also requires planning, testing, debugging, and improving.

Algorithms: The Plan Behind the Code

Before you even write a single line of JavaScript, it helps to plan out your steps in plain English first. This plan is called an **algorithm**: a clear sequence of steps that solves a problem or completes a task, whether or not a computer is involved.

For example, here's an algorithm for deciding whether a player has won a game:

1. Check the player's score.
2. If the score is 500 or higher, the player has won.
3. Otherwise, the game continues.

Notice this algorithm doesn't use any JavaScript syntax at all. It's just a clear, ordered plan. Once you have a solid algorithm, translating it into JavaScript becomes much easier, because you already know exactly what needs to happen.

Tip:

Whenever a coding problem feels overwhelming, stop typing and write out your algorithm in plain English or on paper first. Professional programmers do this constantly. It's often called "pseudocode."

What Is JavaScript, and Why JavaScript?

JavaScript is a programming language created in 1995 by a programmer named Brendan Eich, who built the first version in about ten days. Despite the name, it has nothing to do with the language called Java. The two are about as related as a car and a carpet! JavaScript got its name mostly because Java was very popular at the time.

Here's why JavaScript is a fantastic first language, especially for building games:

- **It runs everywhere:** Every web browser has JavaScript built in, so there is nothing extra to install in order to run your code.
- **It is instantly visible:** You write code, refresh the page, and see the result. That quick feedback makes learning much more fun.

- **It is the language of the web:** Almost every interactive website you have ever used relies on JavaScript.
- **It has graphics built in:** The browser gives JavaScript a drawing surface called the Canvas, which is perfect for games.
- **It is easy to share:** A game written in JavaScript is just a web page, so you can send it to a friend and they can play it right away.

The Three Languages of the Web

Every web page is built from three languages working together. You can think of a web page as a human body:

- **HTML** is the skeleton. It describes what is on the page: headings, paragraphs, buttons, and the canvas.
- **CSS** is the skin and clothing. It controls how things look: colors, sizes, fonts, and layout.
- **JavaScript** is the muscles and brain. It makes things move, react, and think.

In this guide, JavaScript is the star, but you will also learn just enough HTML and CSS to build games. We will introduce them gently as we need them.

What Can You Build With JavaScript?

By the time you finish this guide, you'll have built a game. But JavaScript's uses stretch far beyond games:

- Websites and web applications that react when you click, type, or scroll.
- Mobile apps and desktop apps (many popular apps are built with JavaScript tools).
- Servers and tools that run behind the scenes (using a version of JavaScript called Node.js).
- Interactive art, animations, charts, and data visualizations.
- Browser extensions, bots, and plenty of games!

Setting Up Your Workshop

Every builder needs a workshop. A carpenter needs a workbench and tools; a programmer needs a code editor and a browser. The good news is that you already have the browser. We just need to add an editor.

Step 1: Choose Your Browser

Any modern browser works, but we recommend Google Chrome or Microsoft Edge because their built-in developer tools are excellent and the instructions in this guide will match what you see. Firefox is also a great choice. If you already have one of these installed, you are done with this step.

Step 2: Install a Code Editor

A code editor is a program designed for writing code. It colors different parts of your code, helps you fix typos, and keeps your files organized. We will use **Visual Studio Code** (often called VS Code), which is free and used by professional developers all over the world.

1. Open your browser and search for "Visual Studio Code download."
2. Choose the official download for your computer (Windows, macOS, or Linux).
3. Run the installer and accept the default options.
4. Open VS Code for the first time and choose a color theme, Light or Dark, whichever is easier on your eyes.

Tip:

Ask a parent or teacher to help with downloads and installs. Always download software from the official website so you know it is safe.

Step 3: Make a Project Folder

Every project you build will live in its own folder, like a labeled drawer in a filing cabinet. Let's make one now.

1. On your computer, create a new folder called `js-game-course`.
2. Inside it, create another folder called `week1`.
3. In VS Code, choose File, then Open Folder, and select the `week1` folder.

Step 4: Install Live Server (Optional but Awesome)

Live Server is a free VS Code extension that automatically refreshes your web page every time you save your code. To install it, click the square Extensions icon on the left side of VS Code, search for "Live Server," and click Install. Once it is installed, you can right-click an HTML file and choose "Open with Live Server."

If you would rather not install anything extra, you can simply double-click your HTML file to open it in your browser, then press the refresh button after each change. Both ways work perfectly.

Understanding the VS Code Interface

When you open VS Code with your folder, you will see a few important areas:

- **Explorer (left side):** shows all the files in your project, like a filing cabinet.
- **Editor (middle, the biggest area):** this is where you actually type your code.
- **Line numbers (left edge of the editor):** browser error messages will often mention these numbers, so you can jump straight to the problem.
- **Squiggly underlines:** usually mean a typo or a syntax mistake. Hover over one to see VS Code's suggestion.

Here's What's Happening:

Think of the Explorer as your filing cabinet, the Editor as your notebook where you write, and the browser as the stage where your code performs. Every project you build in this guide will follow this same pattern: write in the editor, watch it run in the browser.

Your First Web Page and Your First JavaScript

Projects in this course use up to three files that work together: `index.html` (the skeleton), `style.css` (the looks), and `script.js` (the brain). For now, we only need two of them.

Creating `index.html`

In VS Code, click the New File icon in the Explorer, name it `index.html`, and type the following code. You can also type an exclamation mark `!` and press Tab to let VS Code write the skeleton for you.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>My First Page</title>
</head>
<body>
  <h1>Hello, programmer!</h1>

  <script src="script.js"></script>
</body>
</html>
```

Here's What's Happening:

`<!DOCTYPE html>` tells the browser "this is a modern web page."

`<html>`, `<head>`, and `<body>` are the three main sections. The head holds information about the page (like its title), and the body holds everything you can see.

`<h1>` creates a big heading. Notice that most HTML tags come in pairs, an opening tag `<h1>` and a closing tag `</h1>`.

The `<script src="script.js">` line is the bridge. It tells the browser, "Please load the JavaScript file named `script.js` and run it." We put it at the bottom of the body so the page is already built by the time our code starts running.

Creating `script.js`

Now create a second file in the same folder called `script.js` and type just one line:

```
console.log("Hello, World!");
```

Open `index.html` in your browser. The page will show your heading, but you won't see "Hello, World!" anywhere on it. That's because `console.log` doesn't write on the page. It writes to a hidden area called the **console**, which is the programmer's secret notepad.

Opening the Developer Console

1. In Chrome or Edge, press F12 (or right-click the page and choose Inspect).
2. Click the tab labeled **Console**.
3. Look for your message: `Hello, World!`

On a Mac, you can also press Command + Option + J in Chrome. If the console is empty, refresh the page and look again.

Here's What's Happening:

`console.log` is a built-in command that prints a message to the console. The word `console` is an object (a toolbox), and `log` is one tool inside it, so we connect them with a dot.

The parentheses () hold whatever we want to print. The text inside quotation marks is called a **string**.

The semicolon ; at the end marks the end of an instruction, like a period at the end of a sentence.

Mini Challenge: Set Up Your First Project

Create the `index.html` and `script.js` files exactly as shown above and open them in your browser.

Change the heading text to include your own name.

Change the console message to something else, such as `console.log("Ready to code!");`, then refresh and find it in the console.

Talking to the Player: alert and prompt

The console is for programmers. To talk to a player, JavaScript gives us two friendly commands:

```
alert("Welcome to the game!");

const name = prompt("What is your name?");
console.log("Nice to meet you, " + name);
```

Here's What's Happening:

`alert("Welcome to the game!")` pops up a message box and pauses everything until the player clicks OK.

`prompt("What is your name?")` pops up a box with a text field. Whatever the player types is handed back to your program.

`const name = ...` stores that answer in a labeled storage box called `name`. (Variables are coming up very soon!)

The last line prints a greeting that sticks the text and the stored name together using the plus sign.

Printing Multiple Things at Once

You can give `console.log` several values separated by commas, and it will print them all on one line with spaces in between:

```
console.log("Score:", 100, "Lives:", 3);
```

```
Score: 100 Lives: 3
```

Mini Challenge: Say Hello Your Way

Write a program that shows an alert with a welcome message for a game you would like to create one day.

Then use `prompt` to ask for the player's name and print "Welcome, [name]!" in the console.

Finally, print three more lines to the console describing your imaginary game: its name, its goal, and its hero.

Variables: Labeled Storage Boxes

A **variable** is a labeled storage box inside the computer's memory. You choose the label (the name) and put a value inside. Later, you can open the box by using its name.

Games are full of things that need remembering: the player's score, how many lives are left, where the spaceship is. Each of those is a variable.

```
let score = 0;
let playerName = "Nova";
const maxLives = 3;

console.log(playerName, "has", score, "points");
```

```
Nova has 0 points
```

Here's What's Happening:

`let score = 0;` creates a new box named `score` and puts the number 0 inside it. The single equals sign `=` is the **assignment operator**. It means "store the value on the right inside the box on the left." It does NOT mean "is equal to" as in math class.

`let playerName = "Nova";` stores a piece of text.

`const maxLives = 3;` creates a box too, but `const` means **constant**: once a value is placed inside, it can never be swapped for a different one.

The final line opens all three boxes and prints what is inside.

let, const, and the Old var

JavaScript has three keywords for making variables:

- **let** is for values that will change, like a score that goes up.
- **const** is for values that should never be reassigned, like the maximum number of lives. If you try to change a `const`, JavaScript will complain with an error, which protects you from accidental mistakes.
- **var** is the old way of doing things. You will see it in older tutorials, but we will not use it in this guide because `let` and `const` are safer.

Tip:

A good habit used by many professional programmers: start with `const` for everything, and only switch to `let` when you discover that the value really needs to change.

Changing a Variable

```
let score = 0;  
score = 10;  
score = score + 5;  
console.log(score);
```

15

Here's What's Happening:

The first line puts 0 in the box.

The second line replaces the 0 with 10. The old value is gone.

The third line is the interesting one. JavaScript always solves the right side first: it opens the `score` box (10), adds 5, getting 15, and then stores that 15 back in the same box. So the box now holds 15.

Naming Rules and Style

Good variable names make your code easy to read, like a clear label on a box. Here are the rules and the common style:

- Names can contain letters, digits, underscores, and the dollar sign, but cannot start with a digit. `player1` is fine, but `1player` is not.
- Names cannot contain spaces or dashes. Use **camelCase**: start with a lowercase word and capitalize each new word, like `playerScore` or `highScore`.

- Names are **case sensitive**: `score` and `Score` are two completely different boxes.
- You cannot use reserved words like `let`, `if`, or `function` as names.
- Pick meaningful names. `lives` is much better than `x`.

Declaring Several Variables

You can create more than one variable on a single line, or leave a variable empty for now:

```
let x = 10, y = 20;  
let enemyCount;  
console.log(x, y, enemyCount);
```

```
10 20 undefined
```

A variable that has been created but not given a value contains the special value `undefined`, which means "nothing has been put in this box yet."

Mini Challenge: Build a Character Sheet

Create variables for a game character: `heroName`, `heroLevel`, `heroHealth`, and `hasShield` (use `true` or `false`).

Print them on separate lines with `console.log`, such as `Name: Nova`.

Level the hero up by increasing `heroLevel` by 1, then print the new level.

Data Types: What Kind of Value Is It?

Every value in JavaScript has a **data type**, which tells the computer what kind of information it is and what you can do with it. You can think of data types as different kinds of containers: a jar for liquids, a box for toys, a bag for groceries.

The Core Data Types

- **String**: text, written between quotation marks. Examples: `"Hello"`, `'Game Over'`.
- **Number**: any number, whole or with decimals. Examples: `42`, `3.14`, `-7`. (Unlike some languages, JavaScript uses one type for both.)
- **Boolean**: only ever `true` or `false`. Booleans are the yes-or-no answers that power game decisions, such as "is the player alive?"
- **Undefined**: the value of a variable that has no value yet.
- **Null**: a value you set on purpose to mean "empty on purpose."

```
let title = "Alien Dodge";
let lives = 3;
let speed = 2.5;
let isGameOver = false;

console.log(typeof title);
console.log(typeof lives);
console.log(typeof speed);
console.log(typeof isGameOver);
```

```
string
number
number
boolean
```

Here's What's Happening:

Each of the four lines at the top stores one kind of value.

typeof is a special word that asks, "What kind of value is this?" and answers with a string.

Notice that both **3** (a whole number) and **2.5** (a decimal) are the same type: number.

Tip:

Here is a famous JavaScript quirk: **typeof null** says **"object"**, which is a mistake that has been part of the language since 1995 and can never be fixed without breaking old websites. Even the best languages have their silly moments!

Strings in Detail

Strings can be written with single quotes, double quotes, or backticks (the key under Esc on most keyboards). Backtick strings are special: they are called **template literals**, and they let you drop variables right into the middle of text using **`${ }`**.

```
const name = "Nova";
const score = 250;

console.log("Player: " + name + ", Score: " + score);
console.log(`Player: ${name}, Score: ${score}`);
```

```
Player: Nova, Score: 250
Player: Nova, Score: 250
```

Here's What's Happening:

The first line builds the message by gluing pieces together with the **+** sign. Joining strings this way is called **concatenation**.

The second line produces exactly the same message, but it is easier to read. Everything inside `${ }` is replaced with its value.

From now on, we'll use template literals whenever we mix text and variables.

A few helpful string tools:

```
const word = "Spaceship";
console.log(word.length);
console.log(word.toUpperCase());
console.log(word.toLowerCase());
console.log(word[0]);
console.log(word.includes("ship"));
```

```
9
SPACESHIP
spaceship
S
true
```

`length` tells you how many characters a string has. `toUpperCase()` and `toLowerCase()` change the case. The square brackets `[0]` grab a single character by its position, and counting always starts at zero. `includes()` checks whether a smaller piece of text appears inside.

Mini Challenge: Type Detective

Create five variables of different kinds: a string, a whole number, a decimal number, a boolean, and one that is created but left empty.

Use `typeof` to print the type of each variable.

Predict the answer before you run the code, then check whether you were right. Which one surprised you?

Comments: Notes to Humans

Computers ignore **comments** completely. They are notes for the humans who read the code, including future you!

```
// This is a single-line comment

/* This is a
   multi-line comment */

let lives = 3; // lives the player starts with
```

Tip:

A good rule of thumb: comments should explain **why** the code does something, not just repeat what it already says. If you had to think for more than a few seconds to understand a piece of code, a short comment will save the next reader (including future you) the same trouble.

You can also use comments to temporarily switch a line off while you test something. Programmers call this "commenting out" a line. In VS Code, select a line and press Ctrl + / (or Command + / on Mac) to toggle a comment.

Doing Math With Operators

JavaScript is a great calculator. The symbols that perform math are called **operators**.

```
console.log(7 + 3); // addition
console.log(7 - 3); // subtraction
console.log(7 * 3); // multiplication
console.log(7 / 2); // division
console.log(7 % 3); // remainder
console.log(2 ** 4); // exponent (power)
```

```
10
4
21
3.5
1
16
```

Here's What's Happening:

The first four lines are the arithmetic you already know. Notice that the multiplication symbol is the asterisk `*`, and division uses the slash `/`.

The `%` symbol is called the **remainder** (or modulo) operator. It gives what is left over after dividing. 7 divided by 3 goes in 2 times with 1 left over, so `7 % 3` is 1. It is surprisingly useful: `number % 2` is 0 for even numbers and 1 for odd numbers.

The `**` operator raises a number to a power, so `2 ** 4` is 2 times 2 times 2 times 2.

Shortcuts for Updating Variables

Games constantly update scores and positions, so JavaScript has shortcuts:

```
let score = 10;
score += 5;    // same as score = score + 5
score -= 2;    // same as score = score - 2
score *= 2;    // same as score = score * 2
score++;       // add 1
score--;       // subtract 1
console.log(score);
```

26

Tip:

JavaScript follows the same order of operations as math class: parentheses first, then exponents, then multiplication and division, then addition and subtraction. When in doubt, add parentheses to make your meaning crystal clear: $(a + b) * c$.

Tip: Fun fact

Try `console.log(0.1 + 0.2)` in the console. It prints `0.30000000000000004`. Computers store decimals in a way that cannot always represent them exactly, and nearly every programming language has this quirk. When you need a tidy number, use `Math.round()` or `.toFixed(2)`.

Getting Input and Converting Types

Here is something that trips up almost every beginner: `prompt()` **always** gives back a string, even when the player types a number.

```
const answer = prompt("Enter a number:");
console.log(answer + 1);
```

If the player types `5`, the output is `51`, not `6`! The plus sign between a string and a number glues them together instead of adding. To fix it, we convert the string into a number. This is called **type conversion** or **type casting**.

```
const answer = Number(prompt("Enter a number:"));
console.log(answer + 1);    // 6
```

```
console.log(Number("42"));    // 42
console.log(Number("3.5"));   // 3.5
console.log(Number("hello")); // NaN
console.log(parseInt("12px")); // 12
console.log(String(99));      // "99"
```

Here's What's Happening:

`Number("42")` turns the text `"42"` into the real number `42`.

`Number("hello")` can't make sense of the word, so it returns `NaN`, which stands for "Not a Number." It is JavaScript's way of saying "that didn't work."

`parseInt("12px")` is more forgiving: it reads whole digits from the start of the text and stops at the first thing that isn't a digit. That is handy for values like "12px" or "100%".

`String(99)` goes the other direction and turns a number into text.

You can check for NaN using `Number.isNaN(value)`.

Simple Mini Projects

Let's combine everything so far into two tiny programs.

Mini Challenge: Greeting Machine

Ask the player for their name and their favorite color using `prompt`.

Show a single `alert` that says something like: "Hello Nova! Blue is a great color." Use a template literal.

Bonus: make the name appear in UPPERCASE.

Mini Challenge: Tip Calculator

Ask for the cost of a meal and the percentage the customer wants to tip. Convert both answers to numbers.

Calculate the tip and the total, and show them with two decimal places using `.toFixed(2)`.

Here is one possible solution for the Tip Calculator, so you can compare it with yours:

```
const bill = Number(prompt("How much was the bill?"));
const tipPercent = Number(prompt("What tip percent?"));

const tip = bill * tipPercent / 100;
const total = bill + tip;

alert(`Tip: ${tip.toFixed(2)}\nTotal: ${total.toFixed(2)}`);
```

Here's What's Happening:

The first two lines ask two questions and immediately convert each answer to a number.

`bill * tipPercent / 100` turns a percentage like 15 into the fraction 0.15 and multiplies it by the bill.

In the final line, `${tip.toFixed(2)}` looks odd but makes sense: the first `$` is just a normal dollar sign, and `${ ... }` fills in the value. The `\n` creates a line break inside the message.

Debugging Basics

Sooner or later, your code will not do what you expected. That is totally normal, and it has a name: a **bug**. Finding and fixing bugs is called **debugging**, and it is one of the most important skills a programmer has.

Reading Error Messages

When JavaScript runs into a problem, it writes a red message in the console. Don't panic! The message is a clue that tells you three things: what kind of error it is, what went wrong, and which file and line number to look at.

```
let playerName = "Nova";  
console.log(playerNme);
```

```
Uncaught ReferenceError: playerNme is not defined  
    at script.js:2
```

Here's What's Happening:

The first word, **ReferenceError**, is the type of problem: we referred to something JavaScript has never heard of.

playerNme is not defined tells us what went wrong: we typed **playerNme** but the variable is called **playerName**. A typo!

script.js:2 tells us the file and the line number, so we know exactly where to look.

The Three Most Common Errors

- **SyntaxError**: you broke a grammar rule, like forgetting a closing quotation mark or bracket. The whole program may refuse to run.
- **ReferenceError**: you used a variable name that doesn't exist, usually because of a typo or a capital letter in the wrong place.
- **TypeError**: you tried to do something a value can't do, like changing a **const** or using a command on **undefined**.

```
const lives = 3;  
lives = 2;
```

```
Uncaught TypeError: Assignment to constant variable.
```

A Simple Debugging Recipe

1. **Read** the error message slowly, especially the line number.
2. **Look** at that line and the line just above it for typos, missing quotes, or missing brackets.
3. **Check** your spelling and capital letters.
4. **Print** with `console.log` to see what is really inside your variables.
5. **Change one thing at a time** and test again.

Tip:

Beginners often change five things at once when something breaks, which makes it impossible to know what fixed it (or what broke it worse!). Change one thing, test, then change the next.

Mini Challenge: Average Score Calculator

Ask the player for three test scores using `prompt()`, converting each one with `Number()`.

Add the three scores together and divide by 3 to find the average.

Show the result in an alert with a template literal, rounded to one decimal place using `.toFixed(1)`.

Week 1 Recap

Let's review everything you've learned this week in one place.

- `console.log()` writes to the developer console; `alert()` and `prompt()` talk to the player.
- Variables are labeled storage boxes. Use `let` for values that change and `const` for values that don't.
- The core data types are string, number, boolean, undefined, and null.
- Comments (`//` and `/* */`) are notes for humans that JavaScript ignores.
- Arithmetic operators (`+`, `-`, `*`, `/`, `%`, `**`) let JavaScript do math, and shortcuts like `+=` and `++` update variables quickly.
- `prompt()` always returns a string, so convert it with `Number()` when you need math.
- Template literals, written with backticks and `${ }`, are the cleanest way to combine text and variables.
- Reading error messages carefully, especially the type of error and the line number, is the first step of debugging.

Check Your Understanding

1. What is the difference between `let` and `const`? When would you choose each one?
2. Why does `prompt()` always give back a string, and how do you turn that string into a number?
3. What will `console.log(typeof 4.0)` display, and why?
4. What is wrong with `console.log("Score: " + 10 + 5)`, if you wanted to see `Score: 15`? How would you fix it?
5. What does the `%` operator calculate, and how could it help you check whether a number is even?

You've officially finished Week 1! You can now store information in variables, understand the basic data types, ask the player questions, do math, and read your first error messages. Every one of these skills will show up again and again as we build our game.

WEEK 2

Control Flow & Loops

Teaching JavaScript to think and repeat itself

Comparison Operators

Last week, your programs always ran the exact same way, top to bottom, every single time. This week, we teach JavaScript how to make decisions: to do one thing if something is true, and a different thing if it's not. Before we can make decisions, though, JavaScript needs a way to compare things.

Comparison operators ask a yes-or-no question and produce a boolean answer: **true** or **false**.

```
const score = 80;

console.log(score === 80); // is it exactly equal to 80?
console.log(score !== 80); // is it NOT equal to 80?
console.log(score > 50);   // greater than
console.log(score < 50);   // less than
console.log(score >= 80);  // greater than or equal to
console.log(score <= 79);  // less than or equal to
```

```
true
false
true
false
true
false
```

Here's What's Happening:

Each line asks a question about **score**, and **console.log** prints the answer.

Notice the **triple** equals sign **===**. One equals sign **=** is for storing a value in a variable. Three equals signs **===** ask the question "are these two things exactly the same?" Mixing these up is one of the most common beginner mistakes!

!== means "is not exactly equal to," and the others work just like the symbols in math class.

Tip:

JavaScript also has a looser comparison with two equals signs **==**, which quietly converts types before comparing. For example, **"5" == 5** is **true**, but **"5" === 5** is **false**. The loose version causes surprising bugs, so in this guide we will always use the strict triple-equals **===** and **!==**.

Making Decisions With if, else if, and else

An **if statement** runs a block of code only when a condition is true. It works just like the rules of everyday life: if it's raining, take an umbrella; otherwise, wear sunglasses.

```
const lives = 0;

if (lives === 0) {
  console.log("Game over!");
} else {
  console.log("Keep playing!");
}
```

```
Game over!
```

Here's What's Happening:

The condition goes inside parentheses after the word **if**. Here, the question is `lives === 0`.

The code inside the curly braces `{ }` is a **block**. It only runs when the condition is true.

The **else** block is the backup plan: it runs when the **if** condition is false.

The spaces at the start of the inner lines are called **indentation**. JavaScript doesn't require them, but they make your code far easier for humans to read, so always use them.

Chaining Decisions

When there are more than two possibilities, use **else if**. JavaScript checks each condition from top to bottom and runs only the first block that matches.

```
const score = 72;

if (score >= 90) {
  console.log("Gold medal!");
} else if (score >= 70) {
  console.log("Silver medal!");
} else if (score >= 50) {
  console.log("Bronze medal!");
} else {
  console.log("Keep practicing!");
}
```

```
Silver medal!
```

Here's What's Happening:

JavaScript tests `score >= 90` first. 72 is not 90 or more, so it skips that block.

Next it tests `score >= 70`. That is true, so it prints "Silver medal!" and then jumps all the way past the rest of the chain. It never even looks at the bronze test.

Because the order matters, we put the hardest condition first. If we tested `score >= 50` first, then a score of 95 would wrongly receive a bronze medal!

Mini Challenge: Password Checker

Use `prompt()` to ask the player for the secret password.

If the answer is exactly the password you chose, show "Access granted!" in an alert. Otherwise, show "Access denied!".

Bonus: if the player's guess has the right letters but the wrong capitalization (like "Banana" instead of "banana"), show a special hint by using `.toLowerCase()` in your comparison.

Boolean Logic: The Brains of the Game

Real games need to check more than one thing at a time. Is the player alive AND holding the key? Is the score high enough OR has the timer run out? JavaScript has three **logical operators** for combining conditions.

- `&&` means **AND**. Both sides must be true.
- `||` means **OR**. At least one side must be true.
- `!` means **NOT**. It flips true to false and false to true.

```
const hasKey = true;
const lives = 2;
const timeUp = false;

console.log(hasKey && lives > 0); // true (both are true)
console.log(lives === 0 || timeUp); // false (neither is true)
console.log(!timeUp); // true (flips false to true)
```

Here's What's Happening:

In the first line, `hasKey` is true and `lives > 0` is true, so AND gives `true`.

In the second line, `lives === 0` is false and `timeUp` is false. OR needs at least one true side, so the result is `false`.

In the third line, `!timeUp` flips `false` into `true`. A handy way to read `!` out loud is "not."

Here is a table that summarizes how AND and OR behave:

A	B	A && B	A B
true	true	true	true
true	false	false	true
false	true	false	true
false	false	false	false

Using Logic in Decisions

```
const hasKey = true;
const bossDefeated = false;

if (hasKey && bossDefeated) {
  console.log("The treasure room opens!");
} else if (hasKey || bossDefeated) {
  console.log("You are making progress...");
} else {
  console.log("Find the key and defeat the boss.");
}
```

```
You are making progress...
```

Truthy and Falsy Values

JavaScript also lets almost any value act like a boolean inside an `if`. These values count as **falsy** (they behave like `false`): `0`, `""` (an empty string), `null`, `undefined`, and `NaN`. Everything else is **truthy**. For example, `if (playerName)` is true whenever the name isn't empty.

Tip:

Think of the logical operators like the rules at a theme park ride: "You must be tall enough AND wearing a seatbelt." Both conditions must be true to ride. But for the fast-lane pass, it might say: "You must have a golden ticket OR a VIP wristband." Only one is needed.

The switch Statement

When you compare one variable against many exact values, a `switch` can be neater than a long chain of `else if` blocks.

```
const choice = prompt("Pick a power-up: speed, shield, or bomb");

switch (choice) {
  case "speed":
    console.log("You run faster!");
    break;
  case "shield":
    console.log("You are protected!");
    break;
  case "bomb":
    console.log("Boom! The screen clears!");
    break;
  default:
    console.log("That power-up doesn't exist.");
}
```

Here's What's Happening:

`switch (choice)` looks at the value stored in `choice`.

Each **case** is one possible match. When JavaScript finds a case that equals the value, it runs the code below it.

The **break** keyword is vital: it says "stop here and leave the switch." Without it, JavaScript would keep running into the next cases below, which is a classic bug.

default is the backup plan, like **else**, for when nothing else matched.

Mini Challenge: Traffic Light Decisions

Ask the player for a traffic light color using `prompt()`: "red," "yellow," or "green."

Use **if**, **else if**, and **else** (or a **switch**) to print "Stop!", "Slow down!", or "Go!" and a different message for anything else.

Bonus: add a second question asking whether any pedestrians are crossing. Use **&&** so that the "Go!" message appears only if the light is green AND no one is crossing.

Loops: Repeating Without Retyping

Imagine you wanted to print "Hello!" one hundred times. You could type the line one hundred times, but that would be silly. A **loop** tells the computer to repeat a block of code for you.

The while Loop

A **while** loop keeps repeating as long as its condition stays true.

```
let countdown = 5;

while (countdown > 0) {
  console.log(countdown);
  countdown--;
}

console.log("Blast off!");
```

```
5
4
3
2
1
Blast off!
```

Here's What's Happening:

Before the loop begins, **countdown** is 5.

JavaScript checks the condition **countdown > 0**. It's true, so it runs the block: it prints 5, then **countdown--** lowers the value to 4.

Then JavaScript goes back to the top and checks the condition again. This continues until **countdown** reaches 0. At that point **0 > 0** is false, the loop ends, and the program moves on to print "Blast off!".

Danger: The Infinite Loop

What would happen if we forgot the line `countdown--`? The value would stay at 5 forever, the condition would never become false, and the loop would run **forever**. This is called an **infinite loop**, and in a browser it freezes the whole tab.

Tip:

If your page freezes because of an infinite loop, don't panic. Close the browser tab (or wait for the browser's "Page Unresponsive" message and choose to close it), fix your code, and open it again. Whenever you write a loop, ask yourself: "What will eventually make this condition false?"

The for Loop

When you know exactly how many times you want to repeat something, the `for` loop packs everything into one tidy line.

```
for (let i = 1; i <= 5; i++) {  
  console.log(`Lap number ${i}`);  
}
```

```
Lap number 1  
Lap number 2  
Lap number 3  
Lap number 4  
Lap number 5
```

Here's What's Happening:

The parentheses after `for` contain three parts separated by semicolons:

`let i = 1` runs once at the very start. It creates a counter named `i` (programmers traditionally use `i` for "index" or "iteration").

`i <= 5` is the condition, checked before every lap. As long as it is true, the loop continues.

`i++` runs at the end of every lap, adding 1 to the counter.

The block in braces is the work to repeat. The first time through, `i` is 1; the second time, 2; and so on.

Skipping and Stopping: `continue` and `break`

- `break` immediately **exits** the loop.
- `continue` skips the rest of this lap and jumps to the next one.

```
for (let i = 1; i <= 10; i++) {
  if (i % 2 === 0) {
    continue;    // skip even numbers
  }
  if (i > 7) {
    break;       // stop the loop completely
  }
  console.log(i);
}
```

```
1
3
5
7
```

Loops Inside Loops

You can put a loop inside another loop. These are called **nested loops**. The inside loop completes all of its laps for every single lap of the outside loop. This idea will return in Week 4 when we build grids and patterns.

```
for (let row = 1; row <= 3; row++) {
  let line = "";
  for (let col = 1; col <= 4; col++) {
    line += "* ";
  }
  console.log(line);
}
```

```
* * * *
* * * *
* * * *
```

Mini Challenge: Countdown and Cheer

Write a **for** loop that counts from 10 down to 1, printing each number.

After the loop, print "Let's play!".

Bonus: use **%** to print "Even" or "Odd" next to each number as it counts down.

Randomness: Making Games Unpredictable

A game where everything happens the same way every time gets boring fast. Randomness makes aliens appear in surprising places, shuffles cards, and rolls dice. JavaScript's built-in **Math** toolbox includes **Math.random()**, which gives a random decimal number from 0 up to (but never including) 1.

```
console.log(Math.random()); // for example: 0.7312845
console.log(Math.random()); // a different number every time
```

To turn this into something useful, we scale it up and round it down using `Math.floor()`, which chops off the decimals.

```
// A random whole number from 1 to 6 (a dice roll)
const dice = Math.floor(Math.random() * 6) + 1;
console.log(`You rolled a ${dice}`);
```

Here's What's Happening:

`Math.random()` gives a decimal such as 0.7312.

Multiplying by 6 stretches the range to 0 up to (but not including) 6, so we might get 4.387.

`Math.floor()` rounds down, giving us a whole number from 0 to 5.

Adding 1 shifts the range to 1 through 6, just like a real die.

A Formula You Can Reuse

To get a random whole number between any `min` and `max` (including both ends):

```
const min = 10;
const max = 20;
const randomNumber = Math.floor(Math.random() * (max - min + 1)) + min;
console.log(randomNumber);
```

This formula will appear in our game to decide where aliens appear. Next week, you will learn to wrap it inside a function so that you can reuse it without retyping it.

Tip:

Other handy `Math` tools: `Math.round(4.6)` gives 5, `Math.ceil(4.1)` rounds up to 5, `Math.abs(-7)` gives 7 (the distance from zero), `Math.max(3, 9, 4)` gives 9, and `Math.PI` is the value of pi. We will use several of these in our game.

Putting It All Together: A Number Guessing Game

Time to combine variables, input, decisions, loops, and randomness into your first real game! The computer thinks of a secret number from 1 to 100, and the player keeps guessing until they get it.

First, let's write the algorithm in plain English:

1. The computer picks a secret number between 1 and 100.
2. Ask the player for a guess.
3. If the guess is too low, say so. If it's too high, say so.
4. Repeat steps 2 and 3 until the guess is right.
5. Tell the player how many guesses it took.

Now the code:

```

const secret = Math.floor(Math.random() * 100) + 1;
let attempts = 0;
let guess = 0;

while (guess !== secret) {
  const answer = prompt("Guess a number from 1 to 100:");

  if (answer === null) {      // the player pressed Cancel
    break;
  }

  guess = Number(answer);
  attempts++;

  if (guess < secret) {
    alert("Too low!");
  } else if (guess > secret) {
    alert("Too high!");
  } else {
    alert(`You got it in ${attempts} tries!`);
  }
}

```

Here's What's Happening:

`secret` holds the random number and never changes, so it uses `const`. `attempts` and `guess` change, so they use `let`.

`while (guess !== secret)` keeps the game running until the guess matches.

`prompt` returns `null` if the player presses Cancel, so we check for that and `break` out of the loop. Without it, the player could never leave!

`guess = Number(answer)` converts the text into a number, and `attempts++` counts the try.

The `if / else if / else` chain gives hints. In the last `else`, the guess must be right, so we celebrate. The loop condition then becomes false and the loop ends.

Tip:

Smart players use a strategy called **binary search**: always guess the middle of the remaining range. Starting at 50, then 25 or 75, and so on, you can always find the answer in 7 guesses or fewer. Computers use the same trick to search through millions of items quickly.

Mini Challenge: Improve the Guessing Game

What happens if the player types the word "banana"? Try it! The guess becomes `NaN`, and the game acts strangely. Fix it using `Number.isNaN(guess)` so the game says "Please type a number" instead. Add a maximum of 7 guesses. If the player runs out, reveal the secret number. Let the player choose the difficulty: easy is 1 to 50, hard is 1 to 1000.

Another Example: Rock, Paper, Scissors

Let's build one more game that practices decisions and randomness. The computer secretly picks one of three options, the player types their choice, and we decide who wins.

```
const roll = Math.floor(Math.random() * 3); // 0, 1, or 2
let computer;

if (roll === 0) {
  computer = "rock";
} else if (roll === 1) {
  computer = "paper";
} else {
  computer = "scissors";
}

const answer = prompt("rock, paper, or scissors?");
const player = answer.toLowerCase().trim();

let result;
if (player === computer) {
  result = "It's a tie!";
} else if (
  (player === "rock" && computer === "scissors") ||
  (player === "paper" && computer === "rock") ||
  (player === "scissors" && computer === "paper")
) {
  result = "You win!";
} else {
  result = "The computer wins!";
}

alert(`You chose ${player}. The computer chose ${computer}. ${result}`);
```

Here's What's Happening:

We turn the random number 0, 1, or 2 into a word using an `if` chain. Declaring `let computer;` before the chain lets us fill it in from any branch.

`answer.toLowerCase().trim()` cleans up the player's typing: it makes everything lowercase and removes extra spaces. Chaining two commands together like this is perfectly normal.

The big middle condition combines `&&` and `||`. Each line in parentheses describes one way the player can win. If any one of them is true, the player wins.

If the choices match it's a tie; if the player didn't win and didn't tie, the computer wins by process of elimination.

Tip:

If the player presses Cancel, `prompt` returns `null`, and `null.toLowerCase()` crashes with a `TypeError: "Cannot read properties of null."` That's a great example of a bug you can discover by testing the way a real player might behave!

Mini Challenge: Improve Rock, Paper, Scissors

Play best-of-three: use a `while` loop and two score variables, `playerScore` and `computerScore`, until someone reaches 2 points.

Check whether the player typed something other than the three valid choices and ask again instead of declaring a winner.

Add "lizard" and "spock" for an extra challenge (hint: write out which options each one beats on paper first).

Week 2 Recap

- Comparison operators (`===`, `!==`, `>`, `<`, `>=`, `<=`) produce a boolean (`true` or `false`).
- `if`, `else if`, and `else` let your program choose between paths. The first matching branch wins.
- Logical operators combine conditions: `&&` (AND), `||` (OR), and `!` (NOT). Falsy values include `0`, `""`, `null`, `undefined`, and `NaN`.
- `switch` compares one value against many exact cases, and each case usually ends with `break`.
- `while` loops repeat while a condition is true, `for` loops are perfect when you know the count, and `break` and `continue` give you control inside a loop.
- Infinite loops freeze the page, so always know what will make a loop stop.
- `Math.random()` creates randomness, and `Math.floor(Math.random() * n)` gives a whole number from 0 to n minus 1.

Check Your Understanding

1. What is the difference between `=` and `===`?
2. In an `else if` chain, why does the order of the conditions matter? Give an example where the wrong order creates a bug.
3. What will `console.log(!true || false)` print, and why?
4. What is an infinite loop, and what is one way to avoid creating one?
5. Write a single line of code that picks a random whole number from 1 to 20.
6. Why does a `switch` case usually end with `break`?

Week 2 is complete! Your programs can now make decisions, repeat actions, and surprise the player with randomness. These are the same ingredients that decide whether an alien appears, whether the player has been hit, and whether the game is over.

WEEK 3

Functions, Data & the DOM

Organizing your code and bringing your web page to life

Functions: Creating Your Own Commands

You've already used built-in functions like `console.log()`, `prompt()`, `Number()`, and `Math.floor()`. A **function** is a named, reusable block of code that does one job. Writing your own functions is like teaching the computer a brand-new word: you explain it once, and then you can use it anywhere, as many times as you like.

```
function greet(name) {  
  console.log(`Welcome, ${name}!`);  
}  
  
greet("Nova");  
greet("Orion");
```

```
Welcome, Nova!  
Welcome, Orion!
```

Here's What's Happening:

The word **function** tells JavaScript, "I'm about to define a new command."

greet is the name we chose for it. Like variables, function names use camelCase and should describe what the function does.

name inside the parentheses is a **parameter**: a placeholder that will be filled in when the function runs. You can think of it as an empty parking space waiting for a car.

The block in braces is the **function body**: the instructions that run each time we use the function.

Writing `greet("Nova")` is called **calling** the function. The value `"Nova"` we hand over is the **argument**, and it parks in the **name** space.

Returning a Value

Some functions do a job and then hand back an answer, like a calculator. The **return** keyword sends a value out of the function.

```
function add(a, b) {  
  return a + b;  
}  
  
const total = add(3, 4);  
console.log(total);  
console.log(add(10, 5) * 2);
```

```
7  
30
```

Here's What's Happening:

`add(a, b)` has two parameters, separated by a comma.

`return a + b` calculates the sum and sends it back to wherever the function was called. The function ends immediately when it hits `return`.

`const total = add(3, 4)` stores the returned value (7) in a variable. And `add(10, 5) * 2` shows that a function call can be used right in the middle of a math expression, just as if it were a number.

Tip:

Don't confuse `return` with `console.log`. Logging only **shows** a value to the programmer.

Returning **hands the value back** so the rest of your program can use it. A function without a `return` gives back `undefined`.

A Function You Already Wrote

Remember the random number formula from Week 2? Let's turn it into a function we can reuse for the rest of the course.

```
function randomInt(min, max) {  
  return Math.floor(Math.random() * (max - min + 1)) + min;  
}  
  
console.log(randomInt(1, 6));      // a dice roll  
console.log(randomInt(100, 500)); // a random x position
```

Arrow Functions and Default Values

JavaScript has a shorter way to write functions, called an **arrow function**. You'll see it everywhere, especially when working with events and lists.

```
const double = (n) => n * 2;

const heal = (health, amount = 10) => {
  return health + amount;
};

console.log(double(8));    // 16
console.log(heal(50));    // 60 (uses the default of 10)
console.log(heal(50, 25)); // 75
```

Here's What's Happening:

`const double = (n) => n * 2;` stores a function in a variable. Everything after the arrow `=>` is the result. When the body is a single expression, the `return` is added automatically.

For longer bodies, we use braces and write `return` ourselves, as in `heal`.

`amount = 10` gives the parameter a **default value**. If you don't supply that argument, JavaScript uses 10 instead.

Mini Challenge: Health Potion Function

Write a function called `drinkPotion(health, potionPower)` that returns the new health after drinking.

Health must never go above 100. (Hint: `Math.min(100, someNumber)` returns whichever number is smaller.)

Test it by calling it with a few different values and printing the results: `drinkPotion(50, 20)` should give 70 and `drinkPotion(90, 40)` should give 100.

Scope: Where Does a Variable Live?

Variables are visible only in the area where they were created. That area is called the variable's **scope**. A variable created inside a function exists only inside that function, like a secret whispered in a closed room.

```
let score = 0;           // global: visible everywhere

function addPoint() {
  const bonus = 1;       // local: only visible inside this function
  score += bonus;
}

addPoint();
console.log(score);      // 1
console.log(bonus);      // ReferenceError: bonus is not defined
```

Variables made with `let` and `const` are also **block scoped**: a variable created inside a pair of curly braces (an `if` block or a `for` loop) stays inside those braces.

Tip:

Beginners often ask, "Why not make everything global so it's always available?" As programs grow, too many global variables get tangled, and any part of the program can change them by accident. Keep variables as local as you reasonably can, and keep only the true game-wide values (like the score and lives) at the top.

Arrays: Lists of Things

A game rarely has just one of something. It has many aliens, many bullets, many items in a backpack. An **array** is an ordered list of values stored in a single variable, written between square brackets.

```
const inventory = ["sword", "shield", "potion"];

console.log(inventory[0]);    // sword
console.log(inventory[2]);    // potion
console.log(inventory.length); // 3

inventory.push("map");        // add to the end
console.log(inventory);        // ["sword", "shield", "potion", "map"]

inventory[1] = "bow";          // replace an item
inventory.pop();               // remove the last item
console.log(inventory);        // ["sword", "bow", "potion"]
```

Here's What's Happening:

Each item in an array has a position number called an **index**, and counting always starts at **0**. So `inventory[0]` is the first item and `inventory[2]` is the third.

`length` tells you how many items the array holds. Notice that the last index is always `length - 1`. `push` adds a new item to the end, `pop` removes the last item, and writing `inventory[1] = "bow"` swaps one item for another.

Interesting detail: we created the array with `const`, yet we changed it. That's allowed! `const` only means the variable can't be pointed at a different array. The contents of the array can still change.

Looping Through an Array

Arrays and loops are best friends. There are three common ways to visit every item:

```
const monsters = ["Slime", "Goblin", "Dragon"];

// 1. A classic for loop with an index
for (let i = 0; i < monsters.length; i++) {
  console.log(`${i}: ${monsters[i]}`);
}

// 2. for...of gives you each item directly
for (const monster of monsters) {
  console.log(monster);
}

// 3. forEach runs a function on each item
monsters.forEach((monster) => {
  console.log(monster.toUpperCase());
});
```

Here's What's Happening:

The classic **for** loop starts **i** at 0 and stops while **i < monsters.length** (not **<=!**), because the last valid index is one less than the length.

for...of is the simplest: on each lap, **monster** is the next item in the list.

forEach hands each item to the arrow function we supply. All three approaches produce the same visits. Use whichever feels clearest.

Useful Array Tools

```
const scores = [40, 15, 90, 62];

console.log(scores.includes(90));      // true
console.log(scores.indexOf(15));       // 1
console.log(scores.join(" - "));       // "40 - 15 - 90 - 62"

const highScores = scores.filter((s) => s >= 50);
console.log(highScores);               // [90, 62]

scores.splice(1, 1);                  // remove 1 item at index 1
console.log(scores);                  // [40, 90, 62]
```

filter is very important for games. It builds a **new** array containing only the items that pass a test. In Week 4, we'll use it to throw away aliens that have left the screen. **splice(index, count)** removes items from the original array.

A random item from an array is easy with our **randomInt** function:

```
const randomMonster = monsters[randomInt(0, monsters.length - 1)];
```

Mini Challenge: Party Roster

Create an array with the names of four adventurers in your party.

Print each name with its position using a loop, like "Member 1: Nova".

Use **push** to recruit a new member and **pop** to lose one. Print the final roster, then pick a random member as the "party leader."

Objects: Describing One Thing in Detail

An array is great for a list of things, but what about describing a single thing with many details? A spaceship has a position, a speed, and a color. An **object** groups related information together using **properties**: pairs of a name (called a key) and a value.

```
const player = {
  name: "Nova",
  health: 100,
  x: 50,
  y: 300,
  speed: 4,
  hasShield: false
};

console.log(player.name);      // Nova
console.log(player["health"]); // 100

player.health -= 10;          // take damage
player.hasShield = true;      // change a value
player.color = "cyan";        // add a brand-new property

console.log(player);
```

Here's What's Happening:

Curly braces `{ }` create an object, and inside we write **key: value** pairs separated by commas.

We read a property with a dot: `player.name`. You can think of it as "the name that belongs to player." The square-bracket style `player["health"]` does the same thing.

We can change a value with `=` or with shortcuts like `-=`, and even add a property that didn't exist before.

Objects are the most natural way to describe game characters: in Week 4, the spaceship and every alien will be objects holding their own **x**, **y**, and **speed**.

Objects Inside Arrays

The most powerful combination in game programming is an **array of objects**: a list where each item is a detailed object.

```
const enemies = [
  { name: "Goblin", hp: 30 },
  { name: "Troll", hp: 80 },
  { name: "Dragon", hp: 200 }
];

for (const enemy of enemies) {
  console.log(`${enemy.name} has ${enemy.hp} HP`);
}

function takeDamage(character, amount) {
  character.hp -= amount;
}

takeDamage(enemies[0], 10);
console.log(enemies[0].hp); // 20
```

Here's What's Happening:

Each pair of braces is one enemy, and the square brackets wrap the whole team.

The loop visits every enemy, and `enemy.name` and `enemy.hp` read properties from the current one.

Our `takeDamage` function receives an object and changes it directly. Because objects are passed by reference (the function gets the real object, not a copy), the change is still there after the function ends. This is exactly how a game will damage the player's ship.

Mini Challenge: Boss Battle

Create an object called `boss` with `name` and `health` (start at 100).

Write a function `attack(target, damage)` that lowers the target's health and prints how much health remains.

Use a `while` loop that keeps attacking with a random damage from 5 to 20 until the boss's health is zero or less, then print "The boss is defeated after X attacks!" (Hint: count attacks in a variable.)

HTML and CSS Essentials for Game Builders

So far, our programs printed to the console or popped up boxes. Real games live on the page itself, so it's time to learn just enough HTML and CSS to build a stage for our code.

HTML in a Nutshell

HTML is made of **elements**, written with tags. Here are the ones we'll use most:

- `<h1>` to `<h6>`: headings, from biggest to smallest.
- `<p>`: a paragraph of text.
- `<div>`: an invisible box used to group and position things.
- `<span>`: a small piece of text inside a line, perfect for a changing number.
- `<button>`: a clickable button.

- **<canvas>**: a drawing surface for graphics and games (Week 4!).

Elements can have **attributes**. The two most important for us are **id** (a unique name for one element) and **class** (a shared name for a group of elements).

```
<h1>Space Shooter</h1>
<p id="scoreboard">Score: <span id="score">0</span></p>
<button id="startBtn">Start</button>
```

CSS in a Nutshell

CSS describes how elements look. A CSS rule has a **selector** (who to style) and a list of **properties** (how to style them).

```
#scoreboard {
  color: white;
  background: #222244;
  font-size: 24px;
  padding: 10px;
  border-radius: 8px;
}

.danger {
  color: red;
}
```

Here's What's Happening:

#scoreboard selects the element whose id is "scoreboard". A hash symbol means id. A dot, as in **.danger**, means class.

color is the text color and **background** is the fill color. Colors can be names (**red**), or hex codes like **#222244**.

font-size and **padding** use **pixels (px)**, tiny dots on the screen. **padding** is the breathing room inside the box, and **border-radius** rounds the corners.

Each declaration ends with a semicolon, and the whole rule sits inside braces.

Linking CSS to Your Page

Put your rules in a file called **style.css**, and connect it in the **<head>** of your HTML file:

```
<link rel="stylesheet" href="style.css">
```

Positioning: Placing Things Exactly

Games need to place things at exact spots. CSS makes this possible with **position**:

- **position: relative** on a parent box makes it the reference frame.

- **position: absolute** on a child lets you place it with **left** and **top**, measured from the corner of the parent.

```
#arena {  
  position: relative;  
  width: 400px;  
  height: 300px;  
}  
  
#target {  
  position: absolute;  
  left: 120px;  
  top: 80px;  
}
```

Tip:

CSS is a rich subject, and you don't need to memorize it. Professional designers look up property names constantly. A search for "CSS center a div" or "CSS rounded corners" will always find an answer.

Mini Challenge: Style a Scoreboard

Build an HTML page with an **h1** title and a scoreboard **div** that has three **span** elements for Score, Lives, and Level.

Create **style.css** and give the scoreboard a dark background, bright text, rounded corners, and some padding.

Make the numbers inside the spans a different color from the labels by giving each a class.

The DOM: JavaScript Meets the Page

When a browser loads an HTML page, it builds a live, tree-like model of every element called the **DOM** (Document Object Model). JavaScript can reach into this model to read, change, add, or remove anything on the page. This is how the page "comes alive."

Finding and Changing Elements

```
<p>Score: <span id="score">0</span></p>  
<button id="addBtn">+1 point</button>
```

```
const scoreEl = document.querySelector("#score");
const addBtn = document.querySelector("#addBtn");
let score = 0;

addBtn.addEventListener("click", () => {
  score++;
  scoreEl.textContent = score;
});
```

Here's What's Happening:

`document.querySelector("#score")` searches the page for the element matching the CSS selector `#score` and hands it back. Because we use the same selector language as CSS, it's easy to remember: `#` for ids, `.` for classes.

We store the found elements in variables so we can use them again quickly.

`addBtn.addEventListener("click", ...)` is how we say, "Whenever this button is clicked, run this function." We call the function an **event handler**.

Inside the handler, `score++` updates our variable, and `scoreEl.textContent = score` rewrites the text on the page. Click the button and watch the number climb!

Changing Styles and Classes

```
scoreEl.style.color = "gold";           // change one style directly
scoreEl.style.fontSize = "32px";        // CSS names become camelCase
scoreEl.classList.add("danger");         // attach a CSS class
scoreEl.classList.remove("danger");      // remove it again
scoreEl.classList.toggle("danger");      // add it if missing, remove if there
```

Tip:

In JavaScript, CSS names with dashes become camelCase: `font-size` becomes `fontSize`, and `background-color` becomes `backgroundColor`. Adding or removing a **class** is usually cleaner than setting many styles one by one, because your CSS file stays in charge of how things look.

Listening to the Keyboard

Games are controlled by key presses, and the browser reports every one as an **event**.

```
document.addEventListener("keydown", (event) => {
  console.log("You pressed:", event.key);
});
```

Open the console, click on the page, and press some keys. You will see names like `a`, `Enter`, `ArrowLeft`, and `" "` (the space bar). Next week, we'll use exactly these key names to steer a spaceship.

Timers: Doing Things Later

Games need to do things after a delay or repeatedly, such as moving a target every second. JavaScript has two timer functions:

```
setTimeout(() => {  
  console.log("Three seconds have passed!");  
}, 3000);  
  
const timerId = setInterval(() => {  
  console.log("Tick");  
}, 1000);  
  
// later, to stop the repeating timer:  
clearInterval(timerId);
```

`setTimeout` runs a function **once** after the number of milliseconds you give (1000 milliseconds is one second). `setInterval` runs it **again and again**, and `clearInterval` stops it. Always keep the timer's id, so that you can stop it.

Mini Challenge: Click Counter

Make a page with a big button labeled "Click me!" and a heading showing the number of clicks.

Each click should add 1 to the counter and update the heading.

Bonus: when the count reaches 10, change the heading color using a CSS class and display "You win!". Another bonus: add a Reset button.

Project: Catch the Bug (Your First Mini Game)

Time to build a complete game using only HTML, CSS, and JavaScript, no canvas needed. Your first game has a clear goal: a bug jumps around an arena, and you score a point every time you click it. You have 20 seconds, and then the game ends.

Step 1: Planning

Let's write the algorithm before writing any code:

1. When the player presses Start, reset the score and the clock, and show the bug.
2. Every 900 milliseconds, move the bug to a new random position.
3. Every second, lower the time left by 1.
4. When the bug is clicked, add 1 to the score and move the bug immediately.
5. When the time reaches 0, stop the timers, hide the bug, and show the final score.

Step 2: The HTML (index.html)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Catch the Bug</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h1>Catch the Bug!</h1>
  <p id="info">
    Score: <span id="score">0</span> | Time: <span id="time">20</span>
  </p>

  <div id="arena">
    <div id="bug"></div>
  </div>

  <button id="startBtn">Start Game</button>

  <script src="script.js"></script>
</body>
</html>
```

Step 3: The CSS (style.css)

The bug itself is drawn using only CSS: a green circle with two black dots for eyes made using the special `::before` and `::after` pieces that every element can have.

```
body {
  font-family: Arial, sans-serif;
  text-align: center;
  background: #1b1b2f;
  color: white;
}

#arena {
  position: relative;
  width: 500px;
  height: 350px;
  margin: 10px auto;
  background: #162447;
  border: 4px solid #4cc9f0;
  border-radius: 12px;
  overflow: hidden;
}

#bug {
  position: absolute;
  left: 225px;
  top: 150px;
  width: 50px;
  height: 50px;
  background: #80ed99;
  border-radius: 50%;
  cursor: pointer;
  display: none;
}
```

```

}

#bug::before,
#bug::after {
  content: "";
  position: absolute;
  top: 12px;
  width: 10px;
  height: 10px;
  background: black;
  border-radius: 50%;
}

#bug::before { left: 10px; }
#bug::after { right: 10px; }

button {
  font-size: 20px;
  padding: 10px 24px;
  border: none;
  border-radius: 8px;
  background: #f72585;
  color: white;
  cursor: pointer;
}

button:disabled {
  background: #555;
  cursor: not-allowed;
}

```

Here's What's Happening:

#arena is the stage. It's **position: relative** so that the bug, which is **position: absolute**, measures its **left** and **top** from the arena's corner. **overflow: hidden** clips anything that pokes outside.

#bug starts with **display: none**, so it's invisible until the game begins. **border-radius: 50%** turns a square into a circle.

#bug::before and **#bug::after** create two extra pieces inside the bug. We make each a small black circle and position them as eyes.

button:disabled styles the button when it is switched off, which we will do while a game is running.

Step 4: The JavaScript (script.js)

```
// 1. Grab the page elements we need
const arena = document.querySelector("#arena");
const bug = document.querySelector("#bug");
const scoreDisplay = document.querySelector("#score");
const timeDisplay = document.querySelector("#time");
const startBtn = document.querySelector("#startBtn");

// 2. Game settings and variables
const GAME_SECONDS = 20;
let score = 0;
let timeLeft = GAME_SECONDS;
let moveTimer = null;
let clockTimer = null;

// 3. Helper function
function randomInt(min, max) {
  return Math.floor(Math.random() * (max - min + 1)) + min;
}

// 4. Move the bug to a random spot inside the arena
function moveBug() {
  const maxX = arena.clientWidth - bug.offsetWidth;
  const maxY = arena.clientHeight - bug.offsetHeight;
  bug.style.left = randomInt(0, maxX) + "px";
  bug.style.top = randomInt(0, maxY) + "px";
}

// 5. Start a new game
function startGame() {
  score = 0;
  timeLeft = GAME_SECONDS;
  scoreDisplay.textContent = score;
  timeDisplay.textContent = timeLeft;

  startBtn.disabled = true;
  bug.style.display = "block";
  moveBug();

  moveTimer = setInterval(moveBug, 900);
  clockTimer = setInterval(tick, 1000);
}

// 6. Runs once every second
function tick() {
  timeLeft--;
  timeDisplay.textContent = timeLeft;

  if (timeLeft === 0) {
    endGame();
  }
}

// 7. Finish the game
function endGame() {
  clearInterval(moveTimer);
  clearInterval(clockTimer);
  bug.style.display = "none";
  startBtn.disabled = false;
}
```

```

startBtn.textContent = "Play Again";
alert(`Time's up! You caught ${score} bugs.`);
}

// 8. Connect the events
bug.addEventListener("click", () => {
    score++;
    scoreDisplay.textContent = score;
    moveBug();
});

startBtn.addEventListener("click", startGame);

```

Here's What's Happening:

Section 1 finds the five elements we need once, at the start, and stores them in constants.

Section 4 is the heart of the movement. The arena's inner width minus the bug's own width is the farthest left the bug can go without sticking out. We use our `randomInt` helper to choose a random spot, then set `left` and `top`. CSS positions need units, so we glue `"px"` onto each number.

Section 5 resets everything, switches the Start button off so the player can't start a second game on top of the first, shows the bug, and starts two timers. We save the timer ids so that we can stop them later.

Section 6 is the clock. It runs every second, lowers `timeLeft`, updates the display, and ends the game at zero.

Section 7 stops both timers, hides the bug, re-enables the button, and announces the final score.

Section 8 connects the events: clicking the bug scores a point and moves it immediately; clicking Start begins the game.

Tip:

Notice how the program is organized into small functions, each doing one job: `moveBug`, `startGame`, `tick`, and `endGame`. When a bug appears, you will usually know exactly which function to look in. This habit becomes even more valuable as our games grow larger.

Mini Challenge: Design Your Own Version

Make the game your own: change the colors, the arena size, and the time limit (remember to update the starting text in the HTML as well).

Make the game harder as the score goes up: for example, every 5 points, stop the move timer and start it again with a shorter delay.

Add a second target, such as a "bomb" that costs the player 3 points when clicked. Draw it with CSS in a different color.

Week 3 Recap

- Functions are reusable, named blocks of code. Parameters are placeholders, arguments are the real values, and `return` sends an answer back.
- Arrow functions `((n) => n * 2)` are a shorter way to write functions.
- Scope decides where a variable can be seen: variables inside a function or block stay inside it.
- Arrays are ordered lists that start counting at index 0. Use `push`, `pop`, `includes`, `filter`, and loops like `for...of` to work with them.
- Objects store related information as key-value pairs, and an array of objects is perfect for groups of game characters.
- HTML builds the skeleton, CSS controls the looks, and the DOM lets JavaScript read and change the page with `querySelector`, `textContent`, `style`, and `classList`.
- Events (`click`, `keydown`) connect player actions to your functions, and timers (`setTimeout`, `setInterval`) make things happen later or repeatedly.

Check Your Understanding

1. What is the difference between a parameter and an argument?
2. What is the difference between a function that uses `console.log` and a function that uses `return`?
3. If an array has 5 items, what is the index of the last item, and why?
4. Why can you still add items to an array that was declared with `const`?
5. How would you write an array of three objects, each describing an alien with an `x` and a `speed`?
6. What does `addEventListener("click", ...)` do, and why do we save a timer's id when we use `setInterval`?

Week 3 is done, and you already finished a real game! You can now organize code into functions, store game data in arrays and objects, and connect JavaScript to a live web page. Next week, we leave the world of page elements behind and meet the powerful drawing tool that every serious browser game uses: the canvas.

WEEK 4

Game Mechanics & Canvas

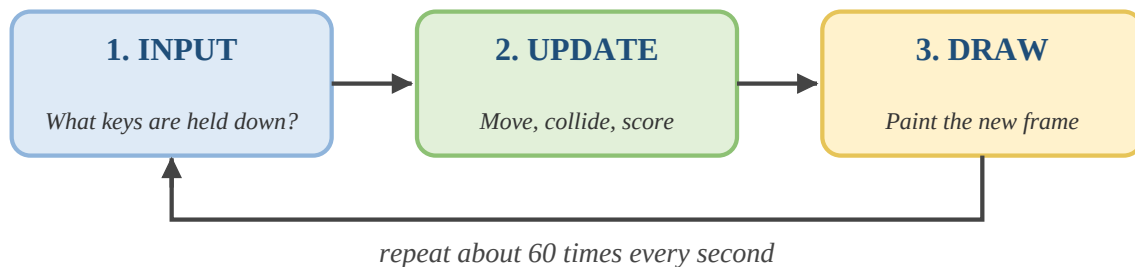
Building Alien Dodge, your first real canvas game

The Core Game Loop

Every video game ever made, from a tiny puzzle on a phone to a giant adventure on a console, is built on the same idea: a cycle that repeats many times every second. Think of a flipbook. Each page shows a slightly different picture, and when you flip through them fast enough, the pictures seem to move. A game works the same way, and every page of the flipbook is called a **frame**.

On every frame, the game does three jobs, always in the same order:

1. **Input:** Check what the player is doing. Which keys are pressed down right now?
2. **Update:** Change the game world. Move the ship, move the aliens, check for crashes, and update the score.
3. **Draw:** Paint the new picture on the screen so the player can see the result.



This cycle is called the **game loop**. A typical screen shows 60 frames every second, so the loop runs about 60 times per second. That's why movement looks smooth. Each time it runs, objects only move a tiny amount.

Tip:

Separating "update" from "draw" is one of the most important habits in game programming. Update changes the numbers (where things are). Draw only looks at the numbers and paints. If you ever catch yourself changing a position inside a draw function, stop. That is a sign the code is in the wrong place.

Planning Our Game: Alien Dodge

This week, we build **Alien Dodge**, a complete game made with HTML Canvas and JavaScript. Here's the pitch: you pilot a small spaceship, and aliens drop from the top of the screen. Dodge them all to win, but each crash costs a life.

The Rules

- The player steers the ship with the arrow keys or the W, A, S, D keys.
- Aliens fall from the top at random positions and speeds.
- If an alien touches the ship, the alien disappears and the player loses 1 life.
- If an alien falls past the bottom edge, the player has dodged it and scores 1 point.
- Reaching 25 points means the player wins. Reaching 0 lives means game over.
- As the score rises, aliens come faster and the game gets harder.

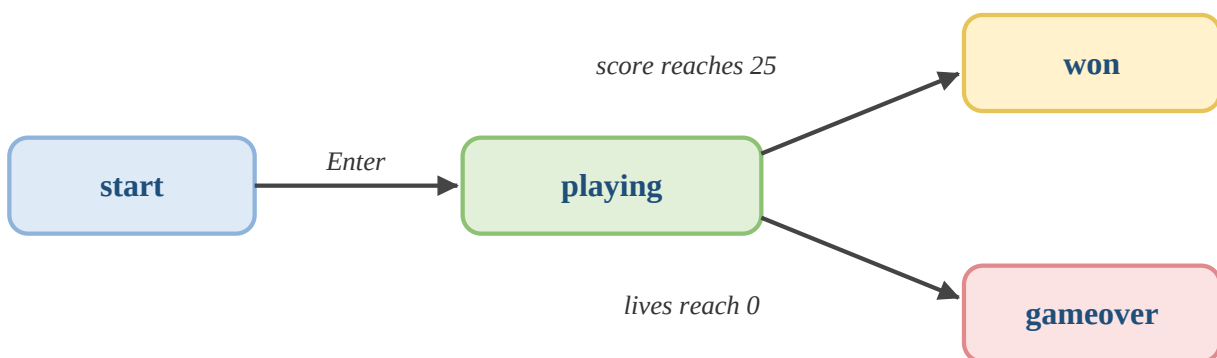
The Game Objects

Remember, objects are perfect for describing things. We need:

- **ship**: an object with `x`, `y`, `radius`, and `speed`.
- **aliens**: an array of objects, each with its own `x`, `y`, `radius`, and `speed`.
- **keys**: an object that remembers which keys are currently held down.

The Game States

A game isn't always running. It also has a title screen and an ending screen. We track which situation we are in with a variable called `state`, which holds one of four words: `"start"`, `"playing"`, `"won"`, or `"gameover"`.



Meet the Canvas

The `<canvas>` element is an empty, rectangular drawing board built into every modern browser. It starts out blank, and JavaScript does all the painting. Nearly every browser game you have ever played is drawn this way.

Setting Up

Create a new folder called `week4`. Inside, make `index.html` with a canvas element:

```
<canvas id="game" width="600" height="500"></canvas>
```

Next, in a JavaScript file, grab the canvas and ask it for a **drawing context**, the object that holds all the painting tools.

```
const canvas = document.querySelector("#game");  
const ctx = canvas.getContext("2d");
```

Here's What's Happening:

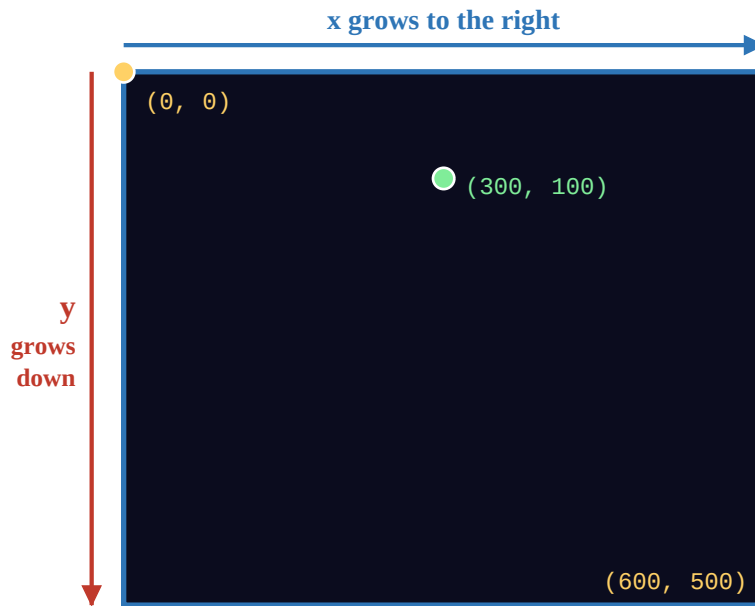
`document.querySelector("#game")` finds the canvas element, just like we found buttons and spans last week.

`canvas.getContext("2d")` hands us a paintbrush for flat, two-dimensional drawing. Programmers nearly always name this variable `ctx`, short for "context." Every drawing command in this lesson is a tool on `ctx`.

The `width` and `height` in the HTML are the size of the canvas in pixels. Set them in the HTML attributes rather than with CSS. Stretching a canvas with CSS makes the drawing blurry.

The Canvas Coordinate System

To draw something, you need to say exactly where. Canvas positions use two numbers: `x` (how far across) and `y` (how far down). Here's the part that surprises everybody: the origin `(0, 0)` is in the **top-left corner**, and `y` increases as you go **down**, the opposite of a math graph.



The origin (0, 0) is the TOP-LEFT corner.

Moving right: add to x.

Moving down: add to y.

Moving up: subtract from y.

Canvas size here: 600 wide, 500 tall.

This matters a lot for games. To make something move down the screen, you **add** to its **y**. To move it up, you **subtract**.

Drawing Rectangles

```
ctx.fillStyle = "tomato";           // choose a fill color
ctx.fillRect(50, 40, 120, 80);      // x, y, width, height

ctx.strokeStyle = "white";          // choose an outline color
ctx.lineWidth = 4;
ctx.strokeRect(50, 40, 120, 80);    // outline of the same rectangle
```

Here's What's Happening:

fillStyle sets the color that will be used by every following fill, until you change it. Think of dipping the brush into a paint can. Colors can be names, hex codes such as `#4cc9f0`, or `rgba(255, 0, 0, 0.5)` for semi-transparent colors.

fillRect(50, 40, 120, 80) paints a solid rectangle whose top-left corner is at `x = 50, y = 40`, and that is 120 wide and 80 tall.

strokeRect draws just the outline, using **strokeStyle** and **lineWidth**.

Drawing Circles

Circles are drawn with a **path**: you start a path, add an arc to it, and then fill it.

```
ctx.fillStyle = "gold";
ctx.beginPath();
ctx.arc(300, 100, 40, 0, Math.PI * 2); // x, y, radius, start angle, end angle
ctx.fill();
```

Here's What's Happening:

`beginPath()` says, "I'm starting a brand-new shape."

`arc(300, 100, 40, 0, Math.PI * 2)` describes a circle centered at (300, 100) with a radius of 40. The last two numbers are angles in **radians**. Going from 0 to `Math.PI * 2` sweeps all the way around, which is a full circle (2 times pi is the equivalent of 360 degrees).

`fill()` paints the shape using the current `fillStyle`.

Drawing Lines and Triangles

Paths can also connect points with straight lines. This is how we'll draw the spaceship.

```
ctx.fillStyle = "#4cc9f0";
ctx.beginPath();
ctx.moveTo(300, 200);    // lift the pen and place it here
ctx.lineTo(270, 260);    // draw a line to here
ctx.lineTo(330, 260);    // and another line to here
ctx.closePath();         // connect back to the start
ctx.fill();
```

Drawing Text

```
ctx.fillStyle = "white";
ctx.font = "bold 24px Arial";
ctx.textAlign = "center";           // "left", "center", or "right"
ctx.fillText("Hello, Canvas!", 300, 350);
```

For text, `x` and `y` mark the **baseline** of the text, roughly where the letters sit, and `textAlign` decides whether `x` is the left edge, center, or right edge.

Erasing

`ctx.clearRect(0, 0, canvas.width, canvas.height)` wipes a rectangle clean, here the whole canvas. A canvas has no memory of shapes. It only holds the final pixels. To animate, you must erase the old picture and paint a fresh one on every frame.

Mini Challenge: Draw a Simple Scene

Fill the whole canvas with a dark blue "night sky" using `fillRect`.

Draw a yellow circle for the moon and a brown rectangle for the ground.

Build a house from a rectangle for the wall, a triangle (use `moveTo` and `lineTo`) for the roof, and a small rectangle for the door.

Bonus: use a loop to draw 20 stars at random positions with the `randomInt` function, each one a tiny white circle.

Game Objects: Building the Ship

Now we apply last week's lessons. The ship is an **object** holding everything we know about it, and a **function** knows how to draw it.

```
const ship = {
  x: 300,      // center of the ship, left to right
  y: 450,      // center of the ship, top to bottom
  radius: 18,  // used to draw AND to detect collisions
  speed: 320   // how many pixels it can travel each second
};

function drawShip() {
  ctx.fillStyle = "#4cc9f0";
  ctx.beginPath();
  ctx.moveTo(ship.x, ship.y - ship.radius);           // nose
  ctx.lineTo(ship.x - ship.radius, ship.y + ship.radius); // left wing
  ctx.lineTo(ship.x + ship.radius, ship.y + ship.radius); // right wing
  ctx.closePath();
  ctx.fill();
}

drawShip();
```

Here's What's Happening:

We store the ship's **center** position in **x** and **y**. Drawing from the center makes later math simpler.

The **radius** is the distance from the center to the edge. We use it to size the triangle now, and later we'll use it to detect crashes.

drawShip reads the ship's numbers and paints a triangle around that center: one point straight up (the nose), one down and to the left, and one down and to the right. If we change **ship.x**, the next time we call **drawShip()** the ship appears somewhere else. That is the secret to all game movement.

Making It Move: The Animation Loop

To animate, we ask the browser to run our function before every screen refresh, using a built-in function called **requestAnimationFrame**.

```

let circleX = 0;

function loop() {
  ctx.clearRect(0, 0, canvas.width, canvas.height); // erase

  circleX += 2; // update
  ctx.fillStyle = "gold"; // draw
  ctx.beginPath();
  ctx.arc(circleX, 100, 20, 0, Math.PI * 2);
  ctx.fill();

  requestAnimationFrame(loop); // ask for the next frame
}

requestAnimationFrame(loop);

```

Here's What's Happening:

`loop` is our game loop in miniature: erase, update, draw.

The very last line inside `loop` asks the browser to call `loop` again before the next screen refresh. That creates a chain that keeps going forever. If you forget that line, the animation draws exactly one frame and stops!

The final line outside the function starts the chain.

Time and Delta Time

There's a hidden problem with `circleX += 2`. A standard monitor refreshes 60 times per second, but a fancy gaming monitor might refresh 144 times per second. On the fancy monitor, the circle would move more than twice as fast! A good game should run at the same speed on every computer.

The fix is to measure how much time has passed since the last frame, called **delta time** (often written `dt`), and to describe speeds in **pixels per second**. Then each frame we move by `speed * dt`.

`requestAnimationFrame` gives our function a timestamp (in milliseconds) each frame, so we can compute `dt` ourselves:

```

let lastTime = 0;

function gameLoop(timestamp) {
  const dt = Math.min((timestamp - lastTime) / 1000, 0.05); // seconds
  lastTime = timestamp;

  update(dt);
  draw();

  requestAnimationFrame(gameLoop);
}

requestAnimationFrame(gameLoop);

```

Here's What's Happening:

`timestamp - lastTime` is the number of milliseconds since the previous frame, and dividing by 1000 turns it into seconds. On a 60 frames-per-second screen, `dt` is about 0.0167.

`Math.min(..., 0.05)` caps `dt` at 0.05 seconds. If the player switches to another browser tab, animation pauses. When they come back, the huge gap could make everything teleport. The cap prevents that.

`update(dt)` and `draw()` are the two halves of every frame. We hand `dt` to `update` so it can scale all movement. For example, a ship with `speed: 320` moves `320 * 0.0167`, about 5.3 pixels, per frame at 60 frames per second. But on a 144 frames-per-second screen, it moves about 2.2 pixels per frame, and the speed ends up exactly the same.

Keyboard Controls

Remember the `keydown` event from Week 3? We'd like to move the ship while a key is held, but there's a catch. If we moved the ship directly inside the `keydown` handler, we would depend on the keyboard's auto-repeat, which has an annoying initial pause. Smooth games use a better trick: **remember which keys are down**, then check that memory on every frame.

```
const keys = {};  
  
document.addEventListener("keydown", (event) => {  
  const key = event.key.toLowerCase();  
  keys[key] = true;  
  
  if (key.startsWith("arrow")) {  
    event.preventDefault();      // stop the page from scrolling  
  }  
});  
  
document.addEventListener("keyup", (event) => {  
  keys[event.key.toLowerCase()] = false;  
});
```

Here's What's Happening:

`keys` starts as an empty object. When a key goes down, we add a property named after the key and set it to `true`. When the key comes back up, we set it to `false`. So `keys["arrowleft"]` always tells us whether the left arrow is held down right now.

We change `event.key` to lowercase so that "A" and "a" (for example, with Caps Lock on) count as the same key, and the arrow keys are named `"arrowleft"`, `"arrowright"`, and so on.

`event.preventDefault()` cancels the browser's normal reaction, which would be to scroll the page when you press an arrow key.

Now the ship can read the memory in its update function:

```
function clamp(value, min, max) {
  return Math.max(min, Math.min(max, value));
}

function updateShip(dt) {
  if (keys["arrowleft"] || keys["a"]) { ship.x -= ship.speed * dt; }
  if (keys["arrowright"] || keys["d"]) { ship.x += ship.speed * dt; }
  if (keys["arrowup"] || keys["w"]) { ship.y -= ship.speed * dt; }
  if (keys["arrowdown"] || keys["s"]) { ship.y += ship.speed * dt; }

  ship.x = clamp(ship.x, ship.radius, WIDTH - ship.radius);
  ship.y = clamp(ship.y, ship.radius, HEIGHT - ship.radius);
}
```

Here's What's Happening:

Each line asks one question: "Is a left key held down?" The `||` lets either the arrow key or the letter key work. If so, we slide the ship by `speed * dt`.

Moving left means subtracting from `x`, and moving up means subtracting from `y`, remember, because `y` grows downward.

`clamp` is a tiny helper: it forces a value to stay between a minimum and a maximum. If `value` is too small, it returns `min`. If too big, it returns `max`. We use it to keep the ship inside the canvas, so the player can't fly away off the screen. We use `ship.radius` as the edge margin so that the whole ship stays visible.

Mini Challenge: Test Your Ship

Put the ship, the keyboard code, and the game loop together on a page (with `WIDTH` and `HEIGHT` constants that match your canvas) and fly the ship around.

Change `ship.speed` to 150, then to 600. How does each value feel? Pick a value you like.

Make the ship move faster while the Shift key is held down. (Hint: the key's name is `"shift"`, so you can multiply the speed when `keys["shift"]` is true.)

Bonus: draw a small orange triangle flame behind the ship when it is moving.

Falling Aliens

Now for the danger. We'll keep all the aliens in an **array of objects**, exactly like the enemies from last week. Every alien has its own position, size, and speed.

Spawning Aliens

```

let aliens = [];
let spawnTimer = 0;      // seconds since the last alien appeared
let spawnDelay = 1;      // seconds between new aliens

function spawnAlien() {
  const radius = randomInt(12, 24);
  aliens.push({
    x: randomInt(radius, WIDTH - radius),
    y: -radius,
    radius: radius,
    speed: randomInt(110, 220) + score * 3,
    done: false
  });
}

```

Here's What's Happening:

Each alien gets a random radius, so some are small and some are big.

x: `randomInt(radius, WIDTH - radius)` chooses a random column that keeps the entire alien inside the screen's width.

y: `-radius` starts the alien just above the top edge, so it glides into view.

The **speed** is a random number plus `score * 3`: the higher the player's score, the faster new aliens fall. That is our first piece of difficulty scaling.

done: `false` is a flag that we'll use to mark aliens for removal.

Moving and Removing Aliens

```

function updateAliens(dt) {
  spawnTimer += dt;
  if (spawnTimer >= spawnDelay) {
    spawnTimer = 0;
    spawnAlien();
  }

  for (const alien of aliens) {
    alien.y += alien.speed * dt;

    if (isColliding(ship, alien)) {
      alien.done = true;      // crashed into the ship
      lives--;
    } else if (alien.y - alien.radius > HEIGHT) {
      alien.done = true;      // fell past the bottom: dodged!
      score++;
    }
  }

  aliens = aliens.filter((alien) => !alien.done);
  spawnDelay = Math.max(0.35, 1 - score * 0.02); // gets harder over time
}

```

Here's What's Happening:

`spawnTimer` collects the time passing, frame by frame. When it reaches `spawnDelay`, we reset it to 0 and spawn a new alien. That is how we create an alien every second or so, without needing `setInterval`.

The loop visits every alien. We move each one down by `speed * dt`, then check two things: has it hit the ship? Or has it fallen completely past the bottom edge (`alien.y - alien.radius > HEIGHT`)?

Instead of deleting an alien from the array while we're in the middle of looping over it, which is risky, we mark it **done**. After the loop, `filter` builds a fresh array that keeps only the aliens that are **not** done.

The last line makes the game faster as the score grows: `spawnDelay` shrinks by 0.02 seconds for each point, but `Math.max` makes sure it never drops below 0.35.

Drawing the Aliens

```
function drawCircle(x, y, radius, color) {
  ctx.fillStyle = color;
  ctx.beginPath();
  ctx.arc(x, y, radius, 0, Math.PI * 2);
  ctx.fill();
}

function drawAliens() {
  for (const alien of aliens) {
    const r = alien.radius;
    drawCircle(alien.x, alien.y, r, "#80ed99"); // body
    drawCircle(alien.x - r * 0.35, alien.y - r * 0.1, r * 0.2, "black");
    drawCircle(alien.x + r * 0.35, alien.y - r * 0.1, r * 0.2, "black");
  }
}
```

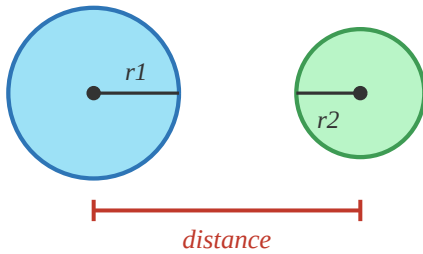
Because we put the circle-drawing steps into a reusable function, drawing an alien takes just three short lines: one green body and two black eyes. Notice that the eyes are sized and placed using fractions of the alien's radius, so they look right on small and large aliens alike.

Collision Detection

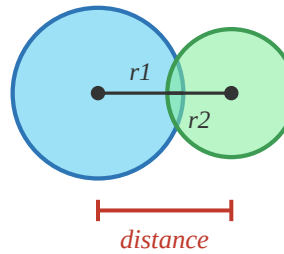
How does the game know that an alien has touched the ship? This is called **collision detection**, and it appears in almost every game. The simplest and fastest method works with circles.

Two circles overlap when the **distance between their centers** is smaller than the **sum of their radii**.

distance > r1 + r2: no collision



distance < r1 + r2: COLLISION!



To find the distance between two points, we use the Pythagorean theorem from math class: a squared plus b squared equals c squared. JavaScript has a handy built-in that does all of that math for us:

`Math.hypot(dx, dy)`.

```
function isColliding(a, b) {  
  const distance = Math.hypot(a.x - b.x, a.y - b.y);  
  return distance < a.radius + b.radius;  
}
```

Here's What's Happening:

`a.x - b.x` is how far apart the two objects are horizontally, and `a.y - b.y` is how far apart they are vertically. These are the two short sides of an imaginary right triangle.

`Math.hypot` returns the length of the long side: the straight-line distance between the two centers.

If that distance is less than `a.radius + b.radius`, the circles must overlap, so the function returns `true`. Otherwise, it returns `false`.

The function works for **any** two objects that have `x`, `y`, and `radius`. In this game, it checks the ship against each alien. In a future game, you could reuse it for bullets, coins, or power-ups.

Tip:

Our ship is drawn as a triangle, but we detect collisions using a circle, a bit like putting an invisible bubble around the ship. This is called a **hitbox**, and it's a trick used by almost every game. It's not perfectly exact, but it's fast, simple, and feels fair to the player. If you ever wonder why a game's collisions feel generous or strict, the hitbox is the answer.

Score, Lives, and Game States

The last big piece is knowing **which phase the game is in**. Our `state` variable controls what updates and what gets drawn.

```

let state = "start";          // "start", "playing", "won", or "gameover"

function update(dt) {
  if (state !== "playing") {
    return;                  // freeze the world unless we're playing
  }
  updateShip(dt);
  updateAliens(dt);
  checkGameEnd();
}

function checkGameEnd() {
  if (lives <= 0) {
    lives = 0;
    state = "gameover";
  } else if (score >= WIN_SCORE) {
    state = "won";
  }
}

```

Here's What's Happening:

At the very top of `update`, a check says, "If we're not in the playing state, leave right away." The `return` keyword exits the function early. That freezes the ship and the aliens on the title screen and on the ending screens.

`checkGameEnd` runs at the end of every playing frame. We check the losing condition first: when `lives` hits zero we switch to `"gameover"`. Otherwise, if the score has reached the target, we switch to `"won"`.

The Enter key moves the game from one phase to the next, and a `resetGame` function sets up a clean new game:

```

function resetGame() {
  score = 0;
  lives = START_LIVES;
  aliens = [];
  spawnTimer = 0;
  spawnDelay = 1;
  ship.x = WIDTH / 2;
  ship.y = HEIGHT - 50;
}

// inside the keydown handler:
if (key === "enter" && state !== "playing") {
  resetGame();
  state = "playing";
}

```

The `draw` function makes decisions based on the same `state` variable: it paints a title screen at the start, the game world while playing, and a darkened overlay with a message when the game is over or won. You'll see it all in the complete listing below.

The Complete Game: Alien Dodge

Here is everything, assembled. Create a folder called **alien-dodge** containing three files. Type them in by hand rather than copying and pasting, if you can. Typing code yourself is the best way to learn it.

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Alien Dodge</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h1>Alien Dodge</h1>
  <canvas id="game" width="600" height="500"></canvas>
  <p>Move with the arrow keys or WASD. Press Enter to start!</p>

  <script src="game.js"></script>
</body>
</html>
```

style.css

```
body {
  background: #10112a;
  color: white;
  font-family: Arial, sans-serif;
  text-align: center;
}

canvas {
  display: block;
  margin: 0 auto;
  background: #0b0c1e;
  border: 4px solid #4cc9f0;
  border-radius: 8px;
}
```

game.js

```
// ===== 1. SET UP THE CANVAS =====
const canvas = document.querySelector("#game");
const ctx = canvas.getContext("2d");
const WIDTH = canvas.width;
const HEIGHT = canvas.height;

// ===== 2. GAME SETTINGS =====
const WIN_SCORE = 25;
const START_LIVES = 3;

// ===== 3. GAME VARIABLES =====
let state = "start"; // "start", "playing", "won", or "gameover"
let score = 0;
let lives = START_LIVES;
let aliens = [];
let spawnTimer = 0; // seconds since the last alien appeared
let spawnDelay = 1; // seconds between new aliens
let lastTime = 0; // timestamp of the previous frame

const ship = { x: WIDTH / 2, y: HEIGHT - 50, radius: 18, speed: 320 };
const keys = {}; // remembers which keys are held down

// ===== 4. HELPER FUNCTIONS =====
function randomInt(min, max) {
  return Math.floor(Math.random() * (max - min + 1)) + min;
}

function clamp(value, min, max) {
  return Math.max(min, Math.min(max, value));
}

function isColliding(a, b) {
  const distance = Math.hypot(a.x - b.x, a.y - b.y);
  return distance < a.radius + b.radius;
}

function resetGame() {
  score = 0;
  lives = START_LIVES;
  aliens = [];
  spawnTimer = 0;
  spawnDelay = 1;
  ship.x = WIDTH / 2;
  ship.y = HEIGHT - 50;
}

// ===== 5. KEYBOARD INPUT =====
document.addEventListener("keydown", (event) => {
  const key = event.key.toLowerCase();
  keys[key] = true;

  if (key.startsWith("arrow")) {
    event.preventDefault(); // stop the page from scrolling
  }

  if (key === "enter" && state !== "playing") {
    resetGame();
    state = "playing";
  }
});
```

```

    }
  });

  document.addEventListener("keyup", (event) => {
    keys[event.key.toLowerCase()] = false;
  });

  // ===== 6. UPDATE: CHANGE THE GAME WORLD =====
  function updateShip(dt) {
    if (keys["arrowleft"] || keys["a"]) { ship.x -= ship.speed * dt; }
    if (keys["arrowright"] || keys["d"]) { ship.x += ship.speed * dt; }
    if (keys["arrowup"] || keys["w"]) { ship.y -= ship.speed * dt; }
    if (keys["arrowdown"] || keys["s"]) { ship.y += ship.speed * dt; }

    ship.x = clamp(ship.x, ship.radius, WIDTH - ship.radius);
    ship.y = clamp(ship.y, ship.radius, HEIGHT - ship.radius);
  }

  function spawnAlien() {
    const radius = randomInt(12, 24);
    aliens.push({
      x: randomInt(radius, WIDTH - radius),
      y: -radius,
      radius: radius,
      speed: randomInt(110, 220) + score * 3,
      done: false
    });
  }

  function updateAliens(dt) {
    spawnTimer += dt;
    if (spawnTimer >= spawnDelay) {
      spawnTimer = 0;
      spawnAlien();
    }

    for (const alien of aliens) {
      alien.y += alien.speed * dt;

      if (isColliding(ship, alien)) {
        alien.done = true; // crashed into the ship
        lives--;
      } else if (alien.y - alien.radius > HEIGHT) {
        alien.done = true; // fell past the bottom: dodged!
        score++;
      }
    }

    aliens = aliens.filter((alien) => !alien.done);
    spawnDelay = Math.max(0.35, 1 - score * 0.02); // gets harder over time
  }

  function checkGameEnd() {
    if (lives <= 0) {
      lives = 0;
      state = "gameover";
    } else if (score >= WIN_SCORE) {
      state = "won";
    }
  }
}

```

```

function update(dt) {
    if (state !== "playing") {
        return;
    }
    updateShip(dt);
    updateAliens(dt);
    checkGameEnd();
}

// ===== 7. DRAW: PAINT THE GAME WORLD =====
function drawCircle(x, y, radius, color) {
    ctx.fillStyle = color;
    ctx.beginPath();
    ctx.arc(x, y, radius, 0, Math.PI * 2);
    ctx.fill();
}

function drawText(text, x, y, size, color, align) {
    ctx.fillStyle = color;
    ctx.font = `bold ${size}px Arial`;
    ctx.textAlign = align;
    ctx.fillText(text, x, y);
}

function drawShip() {
    ctx.fillStyle = "#4cc9f0";
    ctx.beginPath();
    ctx.moveTo(ship.x, ship.y - ship.radius);           // nose
    ctx.lineTo(ship.x - ship.radius, ship.y + ship.radius); // left wing
    ctx.lineTo(ship.x + ship.radius, ship.y + ship.radius); // right wing
    ctx.closePath();
    ctx.fill();
}

function drawAliens() {
    for (const alien of aliens) {
        const r = alien.radius;
        drawCircle(alien.x, alien.y, r, "#80ed99");           // body
        drawCircle(alien.x - r * 0.35, alien.y - r * 0.1, r * 0.2, "black");
        drawCircle(alien.x + r * 0.35, alien.y - r * 0.1, r * 0.2, "black");
    }
}

function drawHUD() {
    drawText(`Score: ${score} / ${WIN_SCORE}`, 15, 30, 22, "white", "left");
    drawText(`Lives: ${lives}`, WIDTH - 15, 30, 22, "white", "right");
}

function drawOverlay(title, color) {
    ctx.fillStyle = "rgba(0, 0, 0, 0.65)";
    ctx.fillRect(0, 0, WIDTH, HEIGHT);
    drawText(title, WIDTH / 2, HEIGHT / 2 - 10, 52, color, "center");
    const mid = WIDTH / 2;
    drawText(`Final score: ${score}`, mid, HEIGHT / 2 + 30, 22, "white", "center");
    drawText("Press ENTER to play again", mid, HEIGHT / 2 + 70, 20, "#ffd166",
        "center");
}

function draw() {
    ctx.fillStyle = "#0b0c1e";
    ctx.fillRect(0, 0, WIDTH, HEIGHT);           // wipe the old frame

```

```

    if (state === "start") {
      const mid = WIDTH / 2;
      drawText("ALIEN DODGE", mid, 190, 52, "#4cc9f0", "center");
      drawText(`Dodge ${WIN_SCORE} aliens to win!`, mid, 240, 22, "white",
        "center");
      drawText("Press ENTER to start", mid, 300, 22, "#ffd166", "center");
      return;
    }

    drawAliens();
    drawShip();
    drawHUD();

    if (state === "won") {
      drawOverlay("YOU WIN!", "#80ed99");
    } else if (state === "gameover") {
      drawOverlay("GAME OVER", "#f72585");
    }
  }
}

// ===== 8. THE GAME LOOP =====
function gameLoop(timestamp) {
  const dt = Math.min((timestamp - lastTime) / 1000, 0.05); // seconds
  lastTime = timestamp;

  update(dt);
  draw();

  requestAnimationFrame(gameLoop);
}

requestAnimationFrame(gameLoop);

```

A Tour of game.js

Here's What's Happening:

Section 1 gets the canvas and the paintbrush, and saves the canvas size in **WIDTH** and **HEIGHT** for easy use everywhere.

Section 2 holds the **settings**: numbers that define the rules. Names written in CAPITAL LETTERS are a programmer tradition for constants that never change.

Section 3 declares every variable the game needs to remember between frames: the current state, score, lives, the aliens, timers, plus the ship and the keys.

Section 4 collects the small helper functions: **randomInt**, **clamp**, **isColliding**, and **resetGame**.

Section 5 listens to the keyboard and fills the **keys** memory.

Section 6 is the **update** half of the game loop. It moves the ship, spawns and moves aliens, tests for collisions, and checks whether the game is over.

Section 7 is the **draw** half. It has no logic that changes the game. It only reads the variables and paints.

Section 8 is the game loop itself, calculating **dt**, calling **update** and **draw**, and asking for the next frame.

Testing Your Game

Open the page and walk through this checklist, like a real tester:

1. The title screen shows, and pressing Enter starts the game.
2. The ship moves with both the arrow keys and W, A, S, D, and stops at all four edges.
3. Aliens fall from random positions and speed up as the score rises.
4. A crash removes one life and removes the alien.
5. Dodged aliens add to the score.
6. Losing all lives shows "GAME OVER," and reaching 25 shows "YOU WIN!"
7. Pressing Enter afterward starts a fresh game, with the score and lives reset.

Tip:

If something doesn't work, don't worry. Open the console (F12) and look for a red error message with a file name and a line number. Check for typos, especially in names like `ship.radius` or `keys["arrowleft"]`. In Week 5, you'll learn a professional debugging process for exactly this situation.

Tuning Your Game

Game designers call the feeling of a game its **game feel**, and it comes from the numbers. Small changes have a big impact. Here are the key dials in our game:

- `ship.speed`: how fast the player can dodge. Higher is easier.
- `START_LIVES`: how forgiving the game is.
- `WIN_SCORE`: how long a game lasts.
- The speed range in `spawnAlien (randomInt(110, 220))`: how fast aliens fall.
- The formula for `spawnDelay`: how quickly the game ramps up the pressure.
- The radius range (`randomInt(12, 24)`): how big and how hard to dodge the aliens are.

A good game is hard enough to be exciting, but fair enough that players believe, "I can do better next time." That feeling is what makes people press Enter and try again.

Mini Challenge: Tweak and Extend Alien Dodge

Make an **Easy** and a **Hard** version by changing only the settings and numbers above. Which settings did you change, and why?

Give every alien a random `color` property when it spawns (for example, picking from an array of three colors), and draw each alien with its own color.

Add a second kind of alien: a small, fast one (radius 8 to 10 and double speed) that appears about one time in five. Hint: use `randomInt(1, 5) === 1` to decide.

Bonus: add a "star" collectible that gives +3 points when the ship touches it instead of costing a life.

Week 4 Recap

- Every game runs a loop of three jobs, input, update, and draw, many times per second.
- The canvas is a drawing board. Its origin `(0, 0)` is at the top-left, and `y` grows downward. We paint with `ctx` using `fillRect`, `arc`, `moveTo` and `lineTo`, and `fillText`, and we erase with `clearRect` or by painting a background.
- `requestAnimationFrame` runs our loop once per screen refresh. Delta time (`dt`) makes the speed the same on every screen, and speeds are measured in pixels per second.
- A `keys` object remembers which keys are held down, and `clamp` keeps objects inside the canvas.
- An array of objects holds the aliens, and `filter` removes the finished ones.
- Circle collision detection compares the distance between centers (`Math.hypot`) with the sum of the radii.
- A `state` variable ("start", "playing", "won", "gameover") controls what the game updates and draws.

Check Your Understanding

1. What are the three jobs a game loop performs on every frame, and in what order?
2. In the canvas coordinate system, which direction does `y` increase, and how would you move an alien down the screen?
3. Why do we multiply movement by `dt`? What could go wrong if we didn't?
4. Why do we store held keys in a `keys` object instead of moving the ship inside the `keydown` handler?
5. Explain, in your own words, how `isColliding` decides whether two circles have touched.
6. Why do we mark aliens as `done` and use `filter` after the loop instead of deleting aliens while looping through them?
7. What would happen to the game if we forgot to call `requestAnimationFrame(gameLoop)` inside `gameLoop`?

Week 4 is complete, and you have built a real, playable, winnable game from scratch, with a game loop, keyboard controls, collisions, scoring, lives, and win and lose screens. Next week is all about taking your game from "working" to "great."

WEEK 5

Polish, Debug & Present

Making your game shine and showing it off

From "It Works" to "It's Great"

Last week, you finished a complete game, and that is a huge achievement. But there's a big difference between a game that works and a game that people love to play. Professional game studios spend as much time on testing, polishing, and fixing as they do on building. This week, you will learn the same skills, and then you will show off your work.

Testing: Try to Break Your Own Game

Testing means running your program on purpose, looking for things that go wrong, before the players do. **Debugging** is what comes next: figuring out why something went wrong and fixing it. Testing finds the bugs, and debugging squashes them.

Playtesting With Fresh Eyes

The best testers think like players who do unexpected things. Try these experiments on Alien Dodge:

- Hold the left and right arrow keys down **at the same time**. What does the ship do?
- Press Enter in the middle of a game. Does anything reset by accident?
- Switch to another browser tab for ten seconds, then come back. Do the aliens jump or teleport?
- Lose on purpose, then restart five times in a row. Does the game behave exactly the same each time?
- Press every key on the keyboard. Does any key break something?
- Make the browser window very small, then very large.

Tip:

When you test your own game, you know exactly what to do, so you play it "correctly" without even noticing. Ask a friend or family member to play without any instructions from you, and just watch quietly. Every time they look confused or press the wrong key, you have found something to improve.

Small Tests for Small Functions

When a function takes some input and returns an answer, such as `clamp` or `isColliding`, you can test it separately from the whole game. A small test for a single function is often called a **unit test**. JavaScript gives us a simple tool for this: `console.assert(condition, message)`. It stays silent when the condition is true, and prints a loud red message when it's false.

```
console.assert(clamp(5, 0, 10) === 5, "clamp keeps 5 between 0 and 10");
console.assert(clamp(-3, 0, 10) === 0, "clamp raises -3 up to 0");
console.assert(clamp(99, 0, 10) === 10, "clamp lowers 99 down to 10");

const near = { x: 0, y: 0, radius: 10 };
const touching = { x: 15, y: 0, radius: 10 };
const far = { x: 25, y: 0, radius: 10 };

console.assert(isColliding(near, touching) === true, "overlapping circles collide");
console.assert(isColliding(near, far) === false, "separate circles do not collide");
```

Here's What's Happening:

Each `console.assert` states something that **must** be true. The first part is the claim, and the second part is the message to show if the claim turns out to be false.

The first three lines check `clamp` with a value inside the range, below the range, and above it. Good tests try the middle **and** the edges.

For `isColliding`, we build three simple circles by hand. The distance from `near` to `touching` is 15, which is less than $10 + 10$, so they should collide. The distance to `far` is 25, which is more than 20, so they shouldn't.

If you run this code and the console stays quiet, every test passed. If you change `clamp` and accidentally break it, a test will tell you immediately.

Mini Challenge: Write Your Own Tests

Add the tests above to the bottom of your `game.js`, temporarily, and check the console. Then deliberately break `isColliding` by changing `<` to `>` and see which tests fail. Fix it again.

Write two more tests: one that checks `randomInt(5, 5)` always equals 5, and one that calls `randomInt(1, 6)` a hundred times in a loop and asserts every result is between 1 and 6.

Remove or comment out your tests when you finish, so they don't clutter the final game.

User Experience: How the Game Feels

User Experience, or UX, is about how it feels to play your game: how easy it is to understand, how fair it feels, and whether the player enjoys it. A game with perfect code can still have a poor UX. Here are the questions game designers ask:

- **Is it clear what to do?** Does the title screen explain the goal and the controls? A player shouldn't need to ask you.

- **Does the game respond?** When something important happens, like a crash, does the player see or hear it right away? This is called **feedback**.
- **Is it fair?** When the player loses, do they feel it was their fault, rather than a trick or a glitch?
- **Is the difficulty right?** The first moments should be easy and welcoming. Challenge should grow gradually. This is the **difficulty curve**.
- **Is it readable?** Is the text large enough? Is there enough contrast between the text and the background? Does the game avoid using color as the **only** way to tell things apart, which helps players who are colorblind?

Tip:

A UX improvement changes how the game feels or how easy it is to understand, like adding on-screen instructions. A bug fix corrects something that is simply broken, like a ship that can leave the screen. They are different kinds of changes, and good games need both.

Polishing Alien Dodge

Let's add five upgrades that make Alien Dodge feel more like a finished game. Each upgrade is separate, so you can add them one at a time, and test after each one. In every step, we show **where** the new code goes.

Upgrade 1: A Scrolling Starfield

A plain background feels empty. A slowly moving field of stars makes the ship feel like it's flying. Add this new array **below** the line `const keys = {};` in Section 3:

```
const stars = [];
for (let i = 0; i < 60; i++) {
  stars.push({
    x: randomInt(0, WIDTH),
    y: randomInt(0, HEIGHT),
    size: randomInt(1, 3)
  });
}
```

Add these two functions to the update and draw sections:

```
function updateStars(dt) {
  for (const star of stars) {
    star.y += star.size * 20 * dt;      // bigger stars move faster
    if (star.y > HEIGHT) {
      star.y = 0;
      star.x = randomInt(0, WIDTH);
    }
  }
}

function drawStars() {
  ctx.fillStyle = "white";
  for (const star of stars) {
    ctx.fillRect(star.x, star.y, star.size, star.size);
  }
}
```

Then call them. Make `updateStars(dt)`; the **first line inside** `update(dt)`, before the check for the playing state, so the stars move on the title screen too. In `draw()`, add `drawStars()`; right after the line that paints the background.

Here's What's Happening:

Each star is an object with a position and a size. We create 60 of them with a loop and `push`.

`star.size * 20 * dt` makes bigger stars drift faster than small ones. This creates the illusion of depth, which is called **parallax**: close things appear to move faster than far things.

When a star falls off the bottom, we place it back at the top at a new random x position, so that we never run out of stars and never create new objects (a good habit for speed).

Upgrade 2: A Red Flash When You Crash

Feedback matters! When the ship is hit, the screen should flash red for a split second.

1. Add a new variable to Section 3: `let hitFlash = 0;`
2. In `resetGame()`, add the line `hitFlash = 0;`
3. In `updateAliens`, right after the line `lives--;`, add `hitFlash = 0.25;`
4. In `update(dt)`, right after `updateStars(dt)`; add this line to make the flash fade away:

```
hitFlash = Math.max(0, hitFlash - dt);
```

1. In `draw()`, right after the line `drawHUD()`; add:

```
if (hitFlash > 0) {
  ctx.fillStyle = `rgba(255, 0, 0, ${hitFlash})`;
  ctx.fillRect(0, 0, WIDTH, HEIGHT);
}
```

Here's What's Happening:

`hitFlash` is a timer measured in seconds. When we crash, we set it to 0.25. Every frame, we subtract `dt` from it, and `Math.max(0, ...)` stops it from going below zero.

We cleverly reuse the timer as the **transparency** of the red overlay. The color `rgba(255, 0, 0, 0.25)` is pure red at 25% strength, and as the number shrinks toward 0, the red fades smoothly away. One variable does two jobs!

Upgrade 3: Pause With the P Key

Every good game lets the player take a break. Add a `paused` flag:

1. Add `let paused = false;` to Section 3, and add `paused = false;` inside `resetGame()`.
2. In the keydown handler, below the `if (key === "enter" ...)` block, add:

```
if (key === "p" && state === "playing") {
  paused = !paused;          // flip between true and false
}
```

1. Change the early-exit check in `update(dt)` so that it also freezes while paused:

```
if (state !== "playing" || paused) {
  return;
}
```

1. In `draw()`, right after the hit-flash block, add:

```
if (paused) {
  ctx.fillStyle = "rgba(0, 0, 0, 0.65)";
  ctx.fillRect(0, 0, WIDTH, HEIGHT);
  const mid = WIDTH / 2;
  drawText("PAUSED", mid, HEIGHT / 2, 52, "#ffd166", "center");
  drawText("Press P to resume", mid, HEIGHT / 2 + 40, 20, "white", "center");
}
```

Here's What's Happening:

`!paused` means "not paused," so `paused = !paused` flips the flag: true becomes false and false becomes true. This is called a **toggle**, and it's a classic way to make a button that switches something on and off.

While `paused` is true, `update` leaves early, so the ship and aliens freeze, but `draw` still runs, so the picture stays on screen with the "PAUSED" message on top.

Upgrade 4: Saving a High Score

Right now, when the player closes the page, everything is forgotten. Browsers provide a small storage area called **localStorage** that keeps data on the player's computer, even after the page is closed. It stores text, saved under a name that you choose.

```
localStorage.setItem("alienDodgeHighScore", 12);           // save
const saved = localStorage.getItem("alienDodgeHighScore"); // load: "12"
```

Notice that `localStorage` always gives text back, just like `prompt` did in Week 1, so we convert it with `Number()`. Also, sometimes a browser blocks storage (for example, in private browsing), which would crash our program. To stay safe, we wrap the risky code in a **try / catch** block: JavaScript **tries** the code, and if an error happens, it jumps to the **catch** block instead of crashing.

Add these new functions to the helper section:

```
function loadHighScore() {
  try {
    return Number(localStorage.getItem("alienDodgeHighScore")) || 0;
  } catch (error) {
    return 0; // storage is blocked: start from zero
  }
}

function saveHighScore() {
  try {
    localStorage.setItem("alienDodgeHighScore", highScore);
  } catch (error) {
    // do nothing: the game works fine without saving
  }
}

function recordScore() {
  if (score > highScore) {
    highScore = score;
    saveHighScore();
  }
}
```

Then connect everything:

1. Add `let highScore = loadHighScore();` to Section 3.
2. In `checkGameEnd`, add the line `recordScore();` inside **both** the `if` branch and the `else if` branch, right after the line that sets the state.
3. In `drawHUD`, add a new line to show the best score in the middle of the top bar:

```
drawText(`Best: ${highScore}`, WIDTH / 2, 30, 22, "#ffd166", "center");
```

Here's What's Happening:

`Number(localStorage.getItem("alienDodgeHighScore")) || 0` does two jobs. If nothing has been saved yet, `getItem` returns `null`, and `Number(null)` is 0. If the stored text were damaged, `Number` would give `NaN`, which is falsy, so the `|| 0` replaces it with 0. This is the "falsy value" idea from Week 2 helping us again.

`try { ... } catch (error) { ... }` means "attempt this, and if it fails, run the fallback instead of crashing." Predictable problems, like blocked storage, are exactly what `try / catch` is for.

`recordScore` runs once when a game ends. If the new score beats the old best, we update `highScore` and save it. Play a game, close the tab, open it again, and your best score is still there!

Upgrade 5: Simple Sound Effects

Browsers include a sound toolkit called the **Web Audio API**. We can use it to create a simple beep from scratch, with no sound files needed.

```
let audioCtx = null;

function beep(frequency, duration) {
  try {
    if (!audioCtx) {
      audioCtx = new AudioContext();
    }
    const osc = audioCtx.createOscillator(); // makes a tone
    const gain = audioCtx.createGain();      // controls the volume
    osc.frequency.value = frequency;
    gain.gain.value = 0.08;                  // keep it quiet
    osc.connect(gain);
    gain.connect(audioCtx.destination);
    osc.start();
    osc.stop(audioCtx.currentTime + duration);
  } catch (error) {
    // no sound available: the game still works
  }
}
```

Now play sounds at key moments: `beep(150, 0.2);` right after `hitFlash = 0.25;` for a low "thud" when you crash, and `beep(880, 0.3);` inside `checkGameEnd` when the player wins.

Here's What's Happening:

An **oscillator** produces a steady tone, and its `frequency` sets the pitch: low numbers like 150 sound like a thud, high numbers like 880 sound like a chime. A **gain** node is a volume knob, set to a quiet 0.08.

We connect the pieces in a chain, oscillator to gain to the speakers (`destination`), then start the tone and schedule it to stop after `duration` seconds.

Browsers only allow sound after the player has interacted with the page, which is fine for us because the player has to press Enter to start. The whole thing is wrapped in `try / catch` so a missing sound system can never crash the game.

Mini Challenge: Make It Yours

Choose at least three more polish ideas and add them: a "NEW HIGH SCORE!" message on the ending screen, an M key that mutes the sound, a different color for the ship while it is flashing, screen text that pulses, or a small ship drawn with more shapes (like a window and flames).

Test your game after **each** change instead of adding everything at once. If something breaks, you will know exactly which change caused it.

Playtest with a friend, and write down two things that confused them.

Debugging Like a Professional

Even professional programmers spend a large part of every day fixing bugs. What makes them good isn't that they avoid bugs, but that they follow a calm, repeatable process.

The Debugging Process

1. **Reproduce it.** Find the exact steps that make the bug appear, every time. A bug you can't repeat is nearly impossible to fix.
2. **Read the error.** Open the console (F12). Is there a red message? Note the error type, the message, and the line number.
3. **Isolate it.** Narrow down the area. Which function is responsible? Comment out other code, or test one part at a time.
4. **Look inside.** Print the values that matter, to see what is **really** happening instead of what you assume.
5. **Fix one thing, then test again.** Changing many things at once hides what actually worked.

Looking Inside With console.log

```
console.log("DEBUG: ship position", ship.x, ship.y);  
console.log("DEBUG: keys", JSON.stringify(keys));  
console.log("DEBUG: alien count", aliens.length);
```

Tip:

Start debugging messages with a word like "DEBUG:" so you can spot them easily in the console, and so you remember to delete them later. Be careful about printing inside the game loop: it runs 60 times per second, which floods the console. Print inside event handlers, or print only when something interesting happens.

Pausing Time With Breakpoints

The browser can freeze your program at a specific line so that you can inspect everything at that exact moment, like pressing pause on a movie.

1. Open the developer tools with F12 and click the **Sources** tab.
2. Find your `game.js` file in the file list on the left and click it.
3. Click on a **line number** to set a breakpoint. A marker appears.
4. Make that line run (for example, press a key). The page freezes and highlights the line.
5. Look at the **Scope** panel on the right to see the current value of every variable, or hover over a variable in the code.
6. Press the Resume button (or F8) to continue, or F10 to run just one line at a time.

You can also write the word `debugger;` directly in your code. When developer tools are open, the program pauses right there.

Classic Bugs in Canvas Games

- **Moving things leave a smear behind them.** You forgot to repaint the background (or call `clearRect`) at the start of each frame.
- **Nothing moves, and there is no error.** You may have forgotten to call `requestAnimationFrame` again inside the loop, or the key name in your code doesn't match the key name you stored.
- **Objects seem to fly off the screen and never come back.** Remember that `y` grows downward, and check whether you are adding when you should be subtracting.
- **The game gets slower and slower.** You might be creating new objects constantly without ever removing old ones. Check that your `filter` runs.
- **Everything is blurry.** You set the canvas size with CSS instead of with the `width` and `height` attributes.
- ****A variable is `undefined` or `NaN`.** ** A property name has a typo, like `ship.raduis`, or you tried to do math on text.

A Bug Hunt, Step by Step

Imagine you edited the keyboard code, and now **the ship won't move left**. There's no red error in the console. Let's follow the process.

1. **Reproduce it:** every time you hold the left arrow, nothing happens. The right arrow works fine.
2. **Read the error:** the console is empty. When there is no error message, the bug is in the logic, not the grammar.
3. **Isolate it:** the ship's movement is in `updateShip`, and the key memory is in the keydown handler.
4. **Look inside:** add `console.log("DEBUG: keys", JSON.stringify(keys));` inside the keydown handler. When you hold the left arrow, it prints `{"arrowleft":true}`.

5. **Compare:** you check `updateShip` and spot the line `if (keys["ArrowLeft"] || keys["a"])`. The memory stores the name in lowercase, `"arrowleft"`, but the code asks for `"ArrowLeft"` with capital letters. That key has never existed in `keys`, so the answer is always `undefined`.
6. **Fix and test:** change it to `keys["arrowleft"]`, remove the debug line, and the ship flies left again.

Tip:

Notice that this process involved no random guessing. We observed what actually happened, formed a specific idea about the cause, and tested that idea. This approach works for bugs of any size, in any programming language.

Mini Challenge: Bug Hunt

Deliberately break your game in three ways, one at a time, and observe what happens:

(1) Remove the line that fills the background at the start of `draw()`. (2) Remove the call to `requestAnimationFrame(gameLoop)` inside `gameLoop`. (3) Change `keys["arrowleft"]` to `keys["ArrowLeft"]`.

For each bug, write down the symptom you saw, whether the console showed an error, and how you found the cause. Then fix all three.

Finally, ask a friend to secretly add one bug to a copy of your game while you aren't looking. Can you find it using the process above?

Sharing Your Game

A game is more fun when other people can play it. Because your game is made of ordinary web files, sharing it is simple:

- **Send the folder.** Put your game's folder into a `.zip` file and send it to a friend or relative. They unzip it and double-click `index.html`.
- **Put it online.** Free hosting services, such as GitHub Pages or Netlify, can turn your folder into a real website with a link that anyone can open. Ask a parent or teacher to help you set up an account, since these services have age rules.
- **Show it live.** Open your game on your computer or a school display and let people try it.

Tip:

When you share your game, keep your personal information out of it: no full name, address, school name, or phone number in the code, the page, or the file names. Also, if you use pictures, sounds, or music made by other people, make sure you have permission, or choose ones that are labeled free to use. The game you built from scratch is safely and completely yours.

Presentation Day

You built a real game, from your very first `console.log("Hello, World!")` all the way to a fully playable, winnable game with scoring, lives, and collision detection. Presentation Day is your chance to show it off with pride.

Here's a simple format for presenting your game to family, friends, or classmates:

1. Name your game and give a one-sentence pitch: what is it, and what's the goal?
2. Demonstrate the controls live, and let your audience try playing it themselves.
3. Share one thing you're especially proud of in your code. Maybe it's the collision detection, the scoring system, or a UX touch you added.
4. Share one bug you found and fixed, and briefly explain how you tracked it down.
5. Share one idea for how you'd expand the game further if you kept working on it.

Tip:

Presenting your work out loud, even to just one person, is a skill in itself, and it's exactly what real software developers do constantly, whether it's showing a manager a new feature or a friend a personal project.

Your Next Game

Alien Dodge is only the beginning. You now know the tools behind thousands of real browser games. Here are five project blueprints. Each one uses skills you have already learned, and each one needs just one or two new ideas. Pick one and build it using either the Canvas or HTML and CSS.

Blueprint 1: Brick Breaker (Canvas)

A ball bounces around, and the player slides a paddle left and right to keep it from falling. The ball breaks bricks stored in an array of objects. The new idea is **bouncing**: when the ball hits a wall, reverse its velocity.

```

const ball = { x: 300, y: 250, radius: 10, vx: 220, vy: 160 };

function updateBall(dt) {
  ball.x += ball.vx * dt;
  ball.y += ball.vy * dt;

  if (ball.x < ball.radius) {
    ball.x = ball.radius;
    ball.vx = Math.abs(ball.vx);    // always head right
  } else if (ball.x > WIDTH - ball.radius) {
    ball.x = WIDTH - ball.radius;
    ball.vx = -Math.abs(ball.vx);   // always head left
  }

  if (ball.y < ball.radius) {
    ball.y = ball.radius;
    ball.vy = Math.abs(ball.vy);    // always head down
  }
}

```

Here's What's Happening:

The ball has a **velocity**: **vx** is its horizontal speed and **vy** is its vertical speed, both in pixels per second. Each frame we move it by velocity times **dt**.

To bounce, we flip the sign of the velocity. When the ball touches the left wall we set **vx** to **Math.abs(ball.vx)**, which is always positive, so the ball heads right. Using **Math.abs** is safer than simply writing **ball.vx = -ball.vx**, which could flip the ball back and forth forever if it were stuck inside a wall.

We also push the ball back inside the wall with **ball.x = ball.radius**, so it never gets stuck.

Blueprint 2: Flappy-Style Flyer (Canvas)

A character falls constantly, and each tap of the space bar makes it flap upward. Gaps in moving pipes (an array of objects) must be passed through. The new idea is **gravity**: a force that changes velocity over time.

```

const bird = { x: 120, y: 250, radius: 14, vy: 0 };
const GRAVITY = 900;    // pixels per second, every second
const FLAP = -320;      // a negative number means upward

function updateBird(dt) {
  bird.vy += GRAVITY * dt;    // gravity speeds up the fall
  bird.y += bird.vy * dt;    // velocity moves the bird
}

document.addEventListener("keydown", (event) => {
  if (event.key === " ") {
    bird.vy = FLAP;           // a flap overrides the fall
  }
});

```

Blueprint 3: Snake (Canvas)

The snake is an array of grid squares. Every 120 milliseconds the game adds a new head in the direction of travel and removes the tail, unless the snake just ate food. The new idea is **grid movement**: instead of smooth motion, positions jump by one cell at a time. Use the timer technique from `spawnTimer` to move the snake at a steady beat. Hint: the head crashes if its position matches any other square in the array.

Blueprint 4: Memory Match (CSS and the DOM)

A grid of face-down cards hides pairs of symbols. Flip two cards: if they match, they stay open, and if not, they flip back after a moment. This game needs no canvas at all. Each card is a `div`, and flipping is a CSS class that you add with `classList.toggle("flipped")`. The new idea is **shuffling** an array so that the pairs are in random places:

```
function shuffle(array) {
  for (let i = array.length - 1; i > 0; i--) {
    const j = randomInt(0, i);
    [array[i], array[j]] = [array[j], array[i]];    // swap two items
  }
  return array;
}

console.log(shuffle(["A", "A", "B", "B", "C", "C"]));
```

Here's What's Happening:

The loop starts at the last item and works backwards. For each position `i`, we choose a random position `j` from 0 up to `i`, then **swap** the items at those two positions.

The strange-looking line `[array[i], array[j]] = [array[j], array[i]]` is a shortcut that swaps two values in one step. It builds a pair on the right side, then unpacks it into the two spots on the left side, in the opposite order.

Use `setTimeout` to flip mismatched cards back after about 800 milliseconds, so the player gets time to remember them.

Blueprint 5: Star Collector (Canvas)

Reuse almost everything from Alien Dodge, with one twist: now some falling objects are **good**. Stars give points when touched, and asteroids cost lives. The new idea is a `type` property on each object ("`star`" or "`asteroid`") that decides how `isColliding` reacts. This is a great way to practice your array of objects and the `filter` method.

Mini Challenge: Plan Your Own Game

Choose one blueprint, or invent your own idea. Write down: the name of the game, its goal, how the player controls it, and how the player wins and loses.

List the game objects you need and the properties each one has. Decide which of them go in arrays. Write the game's algorithm in plain English, then build it step by step, testing after every small change. Start with the smallest possible version that you can play, and add features afterwards.

Week 5 Recap

- Testing means deliberately trying to break your own game to find problems before players do. Small tests of a single function are called unit tests, and `console.assert` lets you write them.
- UX (User Experience) is how a game feels to play, including instructions, feedback, fairness, readability, and a difficulty curve.
- Polish features such as a starfield, a hit flash, a pause button, and sound effects make a game feel finished.
- `localStorage` saves text in the browser even after the page closes, so a high score can survive. Convert saved text with `Number()`.
- `try` and `catch` handle predictable errors gracefully, like blocked storage or missing sound, instead of crashing the whole program.
- Debugging is a repeatable process: reproduce, read the error, isolate, look inside with `console.log` or breakpoints, and fix one thing at a time.
- Presenting your work clearly (the goal, the controls, a proud moment, a fixed bug, and a future idea) is a valuable skill on its own.

Check Your Understanding

1. What is the difference between testing and debugging?
2. Give one example each of a UX improvement and a bug fix in Alien Dodge, and explain why they are different kinds of changes.
3. Why is it helpful to add a temporary `DEBUG:` message with `console.log` instead of just guessing what a variable contains?
4. What does `localStorage.getItem` return when nothing has been saved yet, and why do we convert the result with `Number()`?
5. Why do we wrap the storage and sound code in `try` and `catch` blocks?
6. What does a breakpoint do, and how do you set one in your browser's developer tools?
7. Why is presenting your finished project to someone else a valuable step, even after the code itself is done?

You've reached the end of Week 5, and the end of the course! Look at how far you have come. You started by printing "Hello, World!" and finished with a polished, playable game that saves its high score, and you know how to test it, debug it, and share it.

Quick Reference Cheat Sheet

Keep this page handy while you build. It collects the commands you have used most.

JavaScript Basics

```
let score = 0; // a variable that can change
const MAX_LIVES = 3; // a variable that cannot be reassigned
console.log(`Score: ${score}`); // print with a template literal

if (score >= 10 && lives > 0) { } // decision with AND
for (let i = 0; i < 5; i++) { } // repeat 5 times
while (playing) { } // repeat while true

function add(a, b) { return a + b; } // a function
const double = (n) => n * 2; // an arrow function

const list = [1, 2, 3]; // an array
list.push(4); list.filter((n) => n > 1);
const hero = { name: "Nova", hp: 100 }; // an object
hero.hp -= 10;

Math.floor(Math.random() * 6) + 1; // random whole number, 1 to 6
Math.hypot(dx, dy); // distance
Math.max(0, Math.min(10, value)); // clamp between 0 and 10
```

The DOM, Events, and Timers

```
const el = document.querySelector("#score"); // find an element
el.textContent = "5"; // change its text
el.classList.add("danger"); // add a CSS class
button.addEventListener("click", () => { }); // react to a click
document.addEventListener("keydown", (e) => { console.log(e.key); });
const id = setInterval(tick, 1000); clearInterval(id);
setTimeout(() => { }, 500);
```

Canvas Commands

```
const ctx = canvas.getContext("2d");
ctx.fillStyle = "tomato"; // set the color
ctx.fillRect(x, y, width, height); // solid rectangle
ctx.strokeRect(x, y, width, height); // rectangle outline
ctx.beginPath(); // start a shape
ctx.arc(x, y, radius, 0, Math.PI * 2); // circle
ctx.moveTo(x, y); ctx.lineTo(x, y); // lines
ctx.closePath(); ctx.fill(); // finish and paint
ctx.font = "bold 24px Arial"; ctx.textAlign = "center";
ctx.fillText("Hello", x, y); // text
ctx.clearRect(0, 0, width, height); // erase
requestAnimationFrame(gameLoop); // run the loop next frame
```

Useful Key Names (after toLowerCase)

arrowleft	arrowright	arrowup	arrowdown
enter	escape	shift	" " (the space bar)
a, d, w, s	(letters)	p	(any single letter)

Full Course Glossary

Here is every key term from all five weeks, gathered in one place as a lasting reference you can flip back to any time you need a quick reminder.

- **Algorithm:** a clear, ordered sequence of steps that solves a problem, whether or not a computer is involved.
- **Argument:** the actual value supplied to a function when it's called, matched up with its parameters.
- **Array:** an ordered, changeable list of values, where counting starts at index 0.
- **Arrow function:** a short way to write a function using `=>`.
- **Assignment operator (=):** stores a value inside a variable.
- **Block:** code wrapped in curly braces `{ }`, such as the body of an `if` statement or a loop.
- **Boolean:** a data type that is only ever `true` or `false`.
- **Breakpoint:** a marker in the developer tools that pauses your program at a chosen line so you can inspect it.
- **Bug:** a mistake in code that causes incorrect or crashing behavior. Debugging is the process of finding and fixing bugs.
- **camelCase:** a naming style that starts with a lowercase word and capitalizes each new word, like `highScore`.
- **Canvas:** an HTML element that provides a blank drawing surface, painted with JavaScript.
- **Collision detection:** checking whether two game objects have touched, often using a distance calculation.
- **Comment (`//` or `/* */`):** a note for human readers that JavaScript ignores completely.
- **Comparison operator:** a symbol like `===`, `!==`, `>`, or `<` that compares two values and produces a boolean.
- **Concatenation:** joining strings together, often using the `+` operator.
- **Console:** the area in the developer tools where `console.log` messages and error messages appear.
- **const:** declares a variable that cannot be reassigned.
- **Context (ctx):** the object returned by `canvas.getContext("2d")` that holds all the drawing commands.
- **CSS:** the language that controls how a web page looks: colors, sizes, fonts, and layout.
- **Data type:** the kind of information a value represents: string, number, boolean, undefined, or null.

- **Delta time (dt):** the number of seconds since the previous frame, used to keep game speed the same on every screen.
- **DOM:** the Document Object Model, JavaScript's live model of the elements on a web page.
- **Event:** something that happens that your code can react to, like a click or a key press.
- **Event handler:** a function that runs in response to an event.
- **Frame:** one picture in the sequence of pictures that make up an animation or a game.
- **Function:** a reusable, named block of code, created with the `function` keyword or with an arrow.
- **Game loop:** the repeating cycle of checking input, updating the game, and drawing the screen.
- **Hitbox:** the simple shape, often a circle or rectangle, used to detect collisions with an object.
- **HTML:** the language that describes the structure and content of a web page.
- **Index:** a number representing a position within an array, starting at 0.
- **Infinite loop:** a loop whose condition never becomes false, causing it to repeat forever.
- **let:** declares a variable whose value can change.
- **localStorage:** a small storage area in the browser that keeps text saved even after the page is closed.
- **Logical operator:** `&&`, `||`, or `!`, used to combine or flip boolean conditions.
- **NaN:** "Not a Number," the result of a math or conversion that didn't work.
- **Nested:** a structure (like an `if` or a loop) placed inside another one of the same kind.
- **Object:** a collection of related information stored as key-value pairs called properties.
- **Parameter:** a placeholder name listed in a function's definition, filled in by an argument later.
- **Property:** one named piece of information inside an object, such as `ship.speed`.
- **Return:** a keyword that hands a value back out of a function.
- **Scope:** the part of a program where a variable can be seen and used.
- **State:** a variable that remembers which phase a game is in, such as "playing" or "gameover".
- **String:** text, written between quotation marks or backticks.
- **Template literal:** a string written with backticks that lets you insert variables using `${ }`.
- **Toggle:** switching something between two states, like `paused = !paused`.
- **try / catch:** a way to attempt risky code and run a backup plan if an error occurs.
- **Type conversion:** deliberately converting a value from one data type to another, using `Number()` or `String()`.
- **Unit test:** a small test that checks one function by itself.
- **UX (User Experience):** how a game or app feels to use, including clarity, feedback, fairness, and difficulty.
- **Variable:** a named storage location that holds a value, which can change over time.
- **Velocity:** the speed and direction of a moving object, such as `vx` and `vy`.

Congratulations, and great job finishing this journey! You started five weeks ago not knowing what a variable was, and now you understand variables, data types, conditionals, loops, randomness, functions, arrays, objects, the DOM, events, the HTML canvas, game loops, keyboard controls, collision detection, scoring systems, saved high scores, testing, debugging, and UX. That is a real, professional-level foundation in programming. Keep building, keep experimenting, and keep having fun with code.